

VISVESVARAYA TECHNOLOGICAL UNIVERSITY
“Jnana Sangama”, Belagavi-590014, Karnataka, India



Major Project Phase II Report

On

**“CYBERSECURITY RISK ASSESSMENT IN MEDICAL IOT (MIOT)
NETWORKS”**

Submitted in partial fulfillment of the requirements for the award of the degree Of

Bachelor of Engineering

In

Computer Science and Engineering

Visvesvaraya Technological University, Belagavi.

Submitted by:

VAISHNAVI (1AM22CS216)

SHUBHAM JAISWAL (1AM22CS191)

UMME BARIRA (1AM22CS212)

SHIVA CHARAN REDDY KOTA (1AM22CS184)

Under the Guidance of:

Ms. Snigdha Kesh

Asst Professor, Dept. of CSE



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

AMC ENGINEERING COLLEGE

18th K.M, Bannerghatta Main Road, Bangalore-560083

2026-2027

AMC ENGINEERING COLLEGE

Bangalore-560083

Department of Computer Science and Engineering



CERTIFICATE

This is to certify that the Phase-II project work entitled **“CYBERSECURITY RISK ASSESSMENT IN MEDICAL IOT (MIOT) NETWORKS”** carried out by bonafide students **VAISHNAVI (1AM22CS216), SHUBHAM JAISWAL (1AM22CS191), UMME BARIRA (1AM22CS212) AND SHIVA CHARAN REDDY KOTA (1AM22CS184)** of AMC Engineering College, in partial fulfillment for the award of Bachelor of Engineering in Computer Science & Engineering of Visvesvaraya Technological University(VTU), Belagavi, during the year 2026-2027. It is certified that all corrections and suggestions indicated for internal assessment have been incorporated in the report. The Project Phase-II report has been approved as it satisfies the academic requirements in respect of Project Work Phase-II prescribed for the said Bachelor of Engineering degree.

Project Guide

HOD

Principal

Ms. Shigdhna Kesh
Assistant Professor
Dept. of CSE, AMCEC

Dr. V Mareeswari
Professor & Head
Dept. of CSE, AMCEC

Dr. Yuvaraju B N
Principal
AMCEC

External Viva

External Name

1.

2.

Signature with Date

DECLARATION

We the undersigned students of 7th semester Department of Computer Science & Engineering, AMC Engineering College, declare that our project work entitled **“CYBERSECURITY RISK ASSESSMENT IN MEDICAL IOT (MIOT) NETWORKS”** is a bonafide work of ours. Our project is neither a copy nor by means a modification of any other engineering project. We also declare that this project was not entitled for submission to any other university in the past and shall remain the only submission made and will not be submitted by us to any other university in the future.

NAME	USN	SIGNATURE
VAISHNAVI	1AM22CS216	_____
SHUBHAM JAISWAL	1AM22CS191	_____
UMME BARIRA	1AM22CS212	_____
SHIVA CHARAN REDDY KOTA	1AM22CS184	_____

ACKNOWLEDGMENT

First and foremost we would like to thank GOD, the Almighty for being so merciful on us.

We have a great pleasure in expressing our deep sense of gratitude to founder **Chairman Dr. K.R. Paramahansa** and **Executive Vice President Mr. Rahul Kalluri** for having provided us with a great infrastructure and well-furnished labs for successful completion of our Project.

We express our special thanks and gratitude to our Academic Advisor **Dr. Nagaraja R** for providing us all the necessary advice for successful completion of our project.

We express our sincere thanks and gratitude to our Principal **Dr. Yuvaraju B N** for providing us all the necessary support successful completion of our project.

We would like to extend our special thanks to **Dr.V. Mareeswari** Professor and Head, Department of CSE, for her support and encouragement and suggestions given to us in the course of our project work.

We would like to extend our special thanks to Project coordinator **Mrs. Karthiga G**, Department of CSE, for her support and encouragement and suggestions given to us in the course of our project work

We are grateful to our guide **Ms. Snigdha Kesh**, Asst. Prof., Department of CSE, AMC Engineering College, Bengaluru for her constant motivation & timely help, encouragement and suggestion.

Last but not the least, we wish to thank all the teaching & non-teaching staffs of department of Computer Science and Engineering, for their support, patience and endurance shown during the preparation of this project report.

VAISHNAVI	1AM22CS216
SHUBHAM JAISWAL	1AM22CS191
UMME BARIRA	1AM22CS212
SHIVA CHARAN REDDY KOTA	1AM22CS184

ABSTRACT

The Internet of Things (IoT) is transforming healthcare by enabling remote data storage, continuous patient monitoring, real-time health tracking, and telemedicine for faster diagnosis. Portable sensors and smart medical modules enhance patient outcomes but also expose systems to cybersecurity risks such as data manipulation, denial-of-service, replay, and spoofing attacks, which can threaten patient safety.

The project “Cybersecurity Risk Assessment in IoT Devices” proposes a secure healthcare monitoring system integrating IoT-enabled devices with cloud security and machine learning–based anomaly detection. The system uses an ESP32 microcontroller connected to medical and environmental sensors: MAX30102 for heart rate and SpO₂, DS18B20 for temperature, and BMP180 for pressure and altitude, ideal for patients with respiratory issues.

Data security is ensured through TLS encryption, RSA key exchange, and HMAC-SHA256 hashing for integrity and authenticity. Patient records and device logs are securely stored in Google Firebase Cloud. The system was evaluated under various IoT attacks using a Dynamic Risk Assessment (DRA) framework. Isolation Forest detects abnormal readings and malicious activities in real time. A Streamlit dashboard visualizes vitals and sends alerts for rapid medical response. Overall, the system is lightweight, cost-effective, and secure for home, hospital, and telemedicine use.

ABSTRACT CONTENTS

CERTIFICATE	i
DECLARATION	ii
ACKNOWLEDGEMENT	iii
ABSTRACT	iv
CHAPTER - 1 INTRODUCTION	1-7
1.1 Overview	1-2
1.2 Objectives	2-3
1.3 Purpose, Scope, Applicability	3-7
1.3.1 Purpose	3-4
1.3.2 Scope	4-5
1.3.3 Applicability	6-7
1.4 Organization of Report	7
CHAPTER - 2 LITERATURE SURVEY	8-16
2.1 Introduction	8-9
2.2 Summary of Literature Survey	9-14
2.3 Drawbacks of existing system	14
2.4 Problem statement	15
2.5 Proposed solution	15-16
CHAPTER - 3 REQUIREMENT ENGINEERING	17-21
3.1 Software and Hardware Requirements	17
3.1.1 Software Requirements	17
3.1.2 Hardware Requirements	17
3.2 Conceptual/Analysis Modeling	18-19
3.2.1 Use case diagram	18
3.2.2 Sequence diagram	18
3.2.3 Activity diagram	19
3.3 Software Requirement Specification	20-21
3.3.1 Functional Requirements	20
3.3.2 Non-Functional Requirements	20-21

CHAPTER - 4 PROJECT PLANNING	22-23
1. Project Planning and Scheduling	22-23
CHAPTER - 5 SYSTEM DESIGN	24-30
1. System Architecture	24-25
2. Component Design/Module Decomposition	25-27
3. Interface Design	27
4. Data structure Design	28
5. Algorithm Design	29-30
5.1. Data Acquisition and Security Pre-processing (On ESP32)	29
5.2. Cloud Reception and Integrity Validation (On Firebase Backend)	29
5.3. Anomaly Detection and Alerting (On ML Server / Streamlit)	29-30
CHAPTER - 6 IMPLEMENTATION	31-35
1. Implementation Approaches	31-33
2. Code Snippets	33-35
CHAPTER - 7 TESTING	36-39
1. Definition and Types	36
2. Test Case Analysis	37-38
3. Test Case for Project	38-39
8.1	40-42
Snapshots	2
8.1.1 Hardware Prototype Implementation	40
8.1.2 Real-Time Sensor Data Collection	40
8.1.3 Anomaly Detection	41-42
8.2 Discussion	42
CHAPTER - 8 RESULTS AND DISCUSSION	40-42
CHAPTER - 9 CONCLUSION, APPLICATIONS AND FUTURE WORK	43-44
1. Conclusion	43
2. Applications	43
9.4 Future work	44

CO-PO MAPPING

45

REFERENCES

46-47

ABSTRACT

ANNEXURE A**48-50**

Paper Publication Certificate

48

Protatva-2025 Project Exhibition Certificate

49-50

ABSTRACT**ANNEXURE B****51-52**

KSCST Declaration

51

KSCST Endorsement

52

ABSTRACT LIST OF FIGURES

Figure No.	Figure Name	Page No.
3.1	Use case diagram	18
3.2	Sequence diagram	18
3.3	Activity diagram	19
5.1	Architecture diagram	24
5.2	Component Design	25
5.3	Interface Design	27
6.1	ESP32 Device Firmware (main.cpp)	33
6.2	ESP32 Terminal Output	34
6.3	ML Model Training Script (isolation_forest_model.py)	34
6.4	Data Tampering Attack Script (tamper_attack.py)	35
6.5	Replay Attack Script (replay_attack.py)	35
7.1	Data Travelling from IOT Device to Cloud	37
8.1	System Overview with temperature, (Max130102), BMP180, DS80B20, LCD Sensors	40
8.2	Live Results and Readings	40
8.3	AI Anomaly Detection Graph	41
8.4	Event Logs	41
8.5	Final Dashboard Display	42

ABSTRACT LIST OF TABLES

Table No.	Table Name	Page No.
3.1	Software Requirements	17
3.2	Hardware Requirements	17
5.1	Data Structure Design	28
6.1	Specifications of available tools	31
7.3	Test Case for Project	39

1. Overview

In fact, the Internet of Things (IoT) is revolutionizing healthcare globally through what is known as the Medical Internet of Things (MIoT). The integration of wearable and portable devices with intelligent sensors has significantly enhanced patient care by allowing physicians to remotely monitor patients' health in real time and provide timely diagnostics. The most critical vital signs such as heart rate, oxygen level, body temperature, and environmental conditions can now be continuously monitored and transmitted to cloud platforms. This advancement supports telemedicine, reduces hospital workloads, and ensures quality care beyond clinical settings.

However, despite these advantages, increased interconnectivity has also brought major security risks. Cyber-attackers target healthcare IoT infrastructures due to their sensitive medical data, causing threats such as data falsification, identity spoofing, and denial-of-service (DoS) attacks, which may disrupt patient records, compromise monitoring systems, and even endanger lives. Traditional security measures like authentication protocols and firewalls have proven inadequate against sophisticated attackers. Moreover, while ESP32 microcontrollers are cost-effective and efficient, their limited processing capacity leaves basic security gaps—emphasizing the need for advanced cybersecurity mechanisms integrated within healthcare architectures.

This project introduces a secure healthcare monitoring prototype that integrates IoT hardware, cloud security, and machine learning-based anomaly detection. The system employs an ESP32 microcontroller as the processing core, interfaced with a suite of medical and environmental sensors MAX30102 for heart rate and SpO2, DS18B20 for body temperature, and BMP180 for atmospheric pressure and altitude providing a holistic view of patient health, particularly beneficial for respiratory patients.

Data security is reinforced using Transport Layer Security (TLS) for encrypted communication, RSA for confidentiality, and HMAC-SHA256 hashing to ensure integrity and authenticity. All patient data and logs are securely stored in Google Firebase Cloud to prevent tampering. To evaluate system resilience, we employ a Dynamic Risk Assessment (DRA) framework simulating attacks like flooding, spoofing, and replay under varying conditions, thereby validating robustness against cyber threats.

Machine learning enhances the system's intelligence through the Isolation Forest algorithm, which detects anomalies in both patient data and network traffic. The system identifies abnormal vitals or device malfunctions in real time, alerting doctors for immediate intervention.

For operational convenience, a Streamlit-based dashboard was developed to visualize patient vitals and cyber incidents. It features real-time graphs, instant alerts, and attack simulation capabilities for testing system defenses. With functions to isolate compromised devices, it serves not only as a monitoring interface but also as an active control system enabling healthcare providers to act swiftly during emergencies.

The key innovation of this work lies in merging healthcare monitoring and cybersecurity into one lightweight, portable, and low-cost system. Unlike previous research that handled these areas separately, our approach unifies IoT sensing, cryptography, machine learning, and risk simulation under a single architecture. The outcome is a secure, intelligent, and resilient healthcare framework suitable for home-based monitoring, hospital wards, and telemedicine, offering reliable medical care while defending against modern cyber threats.

2. Objectives

The ultimate aim of this project will be to develop, implement, and evaluate a simulation-based cybersecurity risk assessment framework that specifically targets Medical IoT (MIoT) environments and provide ways to accommodate the dynamic nature of threats and the constraints of healthcare systems. The specific objectives are:

- To develop a structured lightweight simulation-driven risk assessment framework that can be exploited by MIoT systems, and allow active threat detection and manipulation.
- To simulate a variety of cyber-attack circumstances such as, data exfiltration, Denial-of-Service (DoS) attacks, malware infection, and insider threats using Arena® Simulation Software to enhance our understanding of how a medical system reacts under compromised conditions.
- To assess optimum resilience by examining how the healthcare infrastructure, contain and internally return to routine operations during and against cyber-attacks.
- To measure the resource usages of each of the departments and the performance of medial response teams, IT administration staff, and other hospital support teams used during and after active malicious cyber-attacks.
- To assess the integrity and availability of data by understanding how the threats impacted data accuracy, consistency, and access of critical medical data.

- To identify areas of valuable security across the cross-usage of device interactions, data flows, and communication protocols in MIoT-enabled contexts.

3. Purpose, Scope, Applicability

3.1. Purpose

The main purpose of this project is to build a simulation-driven framework that faithfully represents the cybersecurity issues which exist in Medical IoT (MIoT) scenarios. Given the saturation of traditional assessment methods to not faithfully represent the unpredictable, dynamic behavior of MIoT processes, the proposed Dynamic Risk Assessment (DRA) framework is designed as a new, dynamic, proactive and holistic simulation-driven method for understanding some of the cybersecurity issues facing MIoT.

Accurate portrayal of the MIoT cyber threats:

- The DRA framework utilizes discrete-event simulation (DES) to model realistic hospital workflow and medical device interactions in a cyberattack scenario.
- The DRA framework simulates operational breakdowns in real-time as incident(s) occur and also allows analysis on how the system behaves under duress.
- The DRA framework assists healthcare administrators in identifying vulnerabilities of health information systems, devices, and workflows in order to prepare for threats before a cyber attack materializes.

Proactive cyber defense and “what-if” assessment:

- The DRA framework empowers healthcare organizations to simulate several "what-if" scenarios including minor incidents and significant failure in cyber readiness.
- The DRA framework assists healthcare administrators with early identification of failures and weaknesses in the security infrastructure, alongside development of necessary mitigation strategies.
- Therefore, the DRA framework moves from an apparent reactive cyber defensive preparedness position to a proactive, intelligence-based, precautionary cybersecurity posture that supports resilience in healthcare organizations.

Awareness and Education for Non-Technical Stakeholders:

- Offers visual and interactive simulations to show how attacks propagate across MIoT systems.
- Increases the understanding by clinical staff and non-technical users of the importance of cybersecurity best practices.

- Encourages a cybersecurity-aware culture that allows for effective responses and better enforcement of policy.

Incident Response Workflows Optimization:

- Simulates the creation of alerts, availability of resources and when the crisis will be resolved to help manage crises.
- Assists the refinement of emergency procedures, more effective use of staff roles and confirms (for example) if backup systems are operational.
- Enables the continuous improvement of response protocols, with ongoing briefing of emergent scenario processes.

Delivery of a Resilient and Secure Healthcare Experience:

- Ensures that healthcare services that use MIIoT to deliver services are reliable, secure and have a patient focus, whenever cyberthreats occur or are occurring.
- Connects to the long term desire of a hospital to embed cyber risk modeling in health care risk management practices.
- Helps us build a future healthcare system that can accommodate the challenges of complexity in a digital world.

3.2. Scope

This undertaking investigates cybersecurity threats simulated (and analyzed) in Medical Internet of Things (MIIoT) enabled healthcare situations. As hospitals, clinics and remote care become more reliant on smart, connected systems, the implicity of these systems increase their vulnerabilities to crucial network dependencies. The framework establishes inclusive coverage of technological and human factors that relate to cyber vulnerability readiness; by simulating to establish a realistic ecosystem of healthcare operations and effectively fit various risks situations—assisting organizations plan, test and enhancing their cyber secure behaviors.

MIOT Enabled Healthcare Infrastructure Simulation:

- Aims at Cybersecurity risks in smart hospitals, clinics, and remote patient monitoring firmware.
- Models the fundamental technologies such as; sensors, communication technologies .
- Encompasses how these devices interact with human stakeholders.

Coverage of both Technological and Organizational Factors:

- Technological concerned with data flow analysis down to types of systems vulnerabilities and cyber attack methods.
- Organizational models how to allocate resources, human error clusters, alert processing and senior administrative response capabilities.
- Provides assurance that our model derives from what Real healthcare systems do upon cyber threats.

Flexibility Across Healthcare System Sizes:

- Flexible for a variety of health care systems - small clinics to large multi-specialty hospitals.
- Users can configure the simulation model to fit their own operations, and their security issues.

Constructing Realistic Threat Scenarios and Datasets:

- This component focuses on developing synthetic datasets, and cyberattack simulations that accurately depict real MIoT threat vectors.
- The system supports a broad range of test cases that assess readiness at varying levels of risk.

Operable in Real-world Constraints:

- Simulations and the operational landscape must be appropriate for environments that will have limited cybersecurity professionals, regulated environments, or operating in an environment that utilizes legacy systems.
- The simulations will use simple and efficient techniques and there are reusable templates provided that will enable more widespread application.

Used for Validating Performance Measures:

- Simulations are used to validate performance measures.
- Helps assess system behavior under various scenarios.
- Measures include system downtime.
- Evaluates staff workload during incidents.
- Analyzes alert response times.
- Tracks alert resolution effectiveness.
- Checks data integrity under attack conditions.
- Establishes baseline for current security posture.
- Compares new investments to existing performance.
- Ensures alignment with operational objectives.

3.3. Applicability

The Dynamic Risk Assessment (DRA) framework is targeted at meeting the increasing cybersecurity challenges in Medical Internet of Things (MIoT) networks. Importantly, it is applicable across healthcare systems, providing support for operational risk management, strategic planning, medical technology development, policy assessment, and educational activities. The DRA framework utilizes the advantages of a simulated, controlled environment to analyze cyber threats and response strategies. This enables key players to assess a variety of cyber threats for various levels of digital healthcare systems resilience.

Hospitals and Healthcare Organizations:

- Can leverage DRA in regular cybersecurity risk assessment regimes.
- Acts as a virtual test environment for testing the deployment of new MIoT devices.
- Assesses vulnerabilities and tests incident response plans without exposing real infrastructure to threats.

Healthcare Vendors and Medical Device Manufacturers:

- Provides a means to test devices under simulated cyber-attack conditions to see how they react from a security perspective.
- Enhances the security components of products during their design and testing phase.
- Encourages the creation of products that are robust and compliant and meet the expectations of the marketplace and regulators.

Policy and Regulators:

- Allows for impact analysis of alternative compliance regimes (e.g., HIPAA, GDPR).
- Simulates breach of compliance and quantifies the impact on patient data integrity and system reliability.
- Contributes to creating policies that encourage innovation while encoding data security in digital healthcare systems.

Academic and Training Environments:

- Practical tool for students, researchers, and IT staff.
- Enables interactive cyber threat simulations.
- Focuses on MIoT system vulnerabilities.
- Enhances cybersecurity skills and awareness.
- Prepares medical organizations for real attacks.

Cybersecurity Exercises and Training Modules:

- Can be integrated into emergency response training and cybersecurity training exercises performed in hospitals.
- Produces dynamic security risk reports and visual dashboards to help direct response.
- Can assist with decreasing response times, improving playbooks, and generating cybersecurity awareness.

Long-term Viability:

- Designed with flexibility to maintain relevance as the healthcare ecosystem adapts and evolves.
- Able to remain relevant and usable over time because it can incorporate new risks.

4. Organization Report

The project phase-II is systematically organized to provide a comprehensive understanding of the work. Chapter 1 introduces the project, outlining its objectives, purpose, scope, and the organization of the report. Chapter 2 focuses on the literature survey, summarizing existing systems, identifying their drawbacks, and presenting the problem statement alongside the proposed solution. Chapter 3 elaborates on requirement engineering, including software and hardware needs, conceptual modeling, and functional and non-functional requirements. Chapter 4 discusses project planning and scheduling. Chapter 5 delves into system design, covering system architecture, component design, interface design, data structures, and algorithm design. Chapter 6 describes the implementation process and modules, while Chapter 7 evaluates testing strategies and test cases. Finally, Chapter 8 presents results, user interface snapshots, and discussion, and Chapter 9 concludes the report with applications and potential future work.

CHAPTER 2

LITERATURE SURVEY

1. Introduction

The Internet of Medical Things (IoMT) is changing the healthcare landscape by empowering medical devices to communicate with each other, thereby collecting, transmitting, and analyzing patient data ("Data Monetization" 2021). Wearable trackers, implantable sensors, and remote monitoring devices for health and wellness are changing the way we live and minimize the need for patient-doctor visits, diagnostics and treatment, and access to care (with specific distinctions for remote communities). As the pace of digital transformation continues to accelerate, usage of IoMT may very well accelerate.

Though IoMT continues to gain momentum, there are real troubles that come with it. Many IoMT devices have a limited scope of computational resources and therefore, minimally robust security features. If they are not able to connect or require manual intervention, many IoMT devices will remain "offline" - meaning they are very little more than a fancy brick or device that cannot be utilized until someone gets back to the physical property or wrecks it or just chooses not to use it altogether. Ransomware or other cyber-incidents or data exposures provide an abundance of more threats and the remedial actions are never typically when the witnesses appear to have next access to the device. There are indications and proof of patients being exposed to wounds caused by organizational cyber incidents in their electronic medical record or clinical workflows. The biggest issue is that there are multiple devices used in the provision of care, some are more utilized than others for example: telephones, tablets, and mediums to connect to medical devices or phone apps, etc. In this example some medacepts (digital devices) can be linked just through powered up electronics without utilizing the providers facilitated access link which would typically send records of transactional patient-related electronic warrants.

In these circumstances, there have been varied research efforts and offered solutions that might help which include identification of a fuzzy AHP-TOPSIS method for risk approach purposes through risk assessment types, use of lightweight blockchain for secure data exchange, intrusion-detection-systems through AI, or schemes of this sort. It is and remains necessary for any framework to again view these issues using continuous improvement/ hardware limitations, IoMT literature, technical issues and engineering pertaining to the new type of threat from a critical perspective.

Though technology has advanced, there remains a significant gap in cybersecurity communications, especially with patients using implantable or life-dependent devices. Patients are rarely aware of risks to their devices and healthcare providers do not typically have organized ways of informing them. This project is connected to both technology and communication challenges by examining existing frameworks and developing a new strategy to combine and build on frameworks that provide device security and inform patient participation in a more trustworthy and resilient healthcare ecosystem.

2. Summary of Literature Survey

Czekster, R.M.Webber [1] proposed a method "Towards Design and Development of Security Assessment Framework for Internet of Medical Things" provides a fully encompassing stance on initiatives that advance cybersecurity in IoMT surroundings. The authors argue that the vulnerabilities associated with medical devices are increasing because of the greater risks that wearable and implantable technologies face from cyber threats - and those threats are many: ransomware, spoofing, jamming, crypto-jacking and unauthorized firmware update attacks, to name a few. These attacks can jeopardize not only the integrity and confidentiality of sensitive healthcare data - they can also put patient health at risk. Specifically, the authors underscore two key opportunities to implement security measures; the design stage of development and, the post-market. They assert that there are no opportunities to address software vulnerabilities effectively after deployment.

This research presents a security assessment framework that utilizes a fuzzy AHP-TOPSIS method to overcome the challenges presented. This framework evaluates multiple attack surfaces, human, communication, physical, and aggregate, as well as the corresponding fuzzy logic important security attributes, confidentiality, integrity, and availability. Expert survey data with multi-criteria decision-making techniques were analyzed of six various security situations. The research findings demonstrate the framework's effectiveness in identifying the least-secritised IoMT, and as a guide systematic tool for stakeholders in identifying security methods to assist with cybersecurity decision-making during the entire lifecycle of medical devices and healthcare systems.

Si-Ahmed, A., Al-Garadi [2] In "A Lightweight Blockchain and Fog-Enabled Secure Remote Patient Monitoring System," the authors propose a lightweight, secure, and efficient architecture that integrates the Internet of Things (IoT), blockchain, and fog computing to provide extensive support for remote patient monitoring (RPM).

This paper addresses the challenge of protecting sensitive medical information obtained from wearable devices; it proposes a secure, lightweight blockchain, complemented by a fog layer to allow rapid, local execution of consent, access control, and security processing for data obtained from remote health monitoring devices. The architecture is motivated by the need to reduce latency from the medical wearable device to the data repository (cloud) for real-time alerts, and to develop a lightweight consensus protocol that is compatible with low-power IoT devices.

The architecture presents a three-layer model. The IoT layer is responsible for collecting data from remote health monitoring devices (sensors); the fog layer is responsible for processing the data in real-time (low latency) and the cloud layer is responsible for storing the data in the long-term (for record retention). The architecture leverages a local blockchain and a global blockchain to support smart contracts for access control and data integrity. The performance results show up to 40% lower average latency, 36% less energy usage, and 50% faster transaction retrieval (after latency) than cloud-only RPM systems. The architecture can support real-time alerts that are sufficient for a RPM system. Additionally, the architecture has also been shown to be resilient to non-destructive cyberattacks to the system, which will provide assurance to managers that the framework can not only deliver reliable but now secure healthcare services.

AlSalem, T.S., Almaiah, M.A., & Lutfi [3] proposed a method for emphasize the value of patient involvement in the cybersecurity of their medical devices. The technical world is always progressing, while the patient education space continues to lag. Patients are isolated from understanding how their devices operate as well as how the data is handled and the potential risks of transmission. For patients to make informed decisions about their health care, they must first understand this fundamental awareness of device safety.

The authors recommend the creation of better education strategies and communication efforts to address this issue, especially for sensitive populations such as older adults. Future research to develop various patient-specific cybersecurity awareness programs as well as other device-specific programs should consider other populations and types of devices.

ElSayed, Z., Abdelgawad, A., & Elsayed.N [4] The paper "Trust and Risk Assessment in IoT Networks" proposes an architecture for a security framework based on combining device fingerprinting and dynamic trust to evaluate risk levels in IoT devices. The new architecture appears not only scalable, but it addresses the limitations of implementing traditional toolsets for today's level of device connectivity.

Operationally, the new system performs device behaviour monitoring for packet clock timing, flow of data etc and correlates device behaviour with dynamic trust over time to assign trust scores to elements of interest. The presence of any abnormal activity for any device while being monitored reduces trust, this value is combined with a static profile of the device to create a complete risk score when considering the total environment being monitored.

The proposed device risk framework included an IoT gateway with an introduced meta-device which was described as a "trust daemon". The trust daemon operated empirically, monitoring devices as they went about their behaviour in a passive manner and categorising devices into risk classifications of low, medium, or high. Further explanation showed that devices with higher capabilities (processing power/memory) invoked a higher priority level for behaviour violations in the supervision process should they deviate into another risk classification. The tests that followed showed the framework was effective at monitoring and detecting anomalous behaviour regardless of the device type being operated, with respect to the limitations of devices fingerprinted all of which produced occasional errors in the framework. In totality, the adaptive risk based framework proposed creates a scalable approach to security specifically in environments that are complex like an IoT environment, while remaining thorough and efficient in its security overwhelming.

Hemmes, J., Fulton, S., Dressler, J., & Kirkman, S.[5] proposed an intelligent approach on "Cybersecurity Risk Analysis in the IoT: a Systematic Review" analyzes 40 recent studies to identify common cybersecurity challenges related to IoT security, such as privacy violations, cybercrimes, and denial-of-service attacks. Although AI and machine learning applications have the potential for threat detection, issues such as server breaches and weak user authentication are still prevalent.

Emerging trends include using AI as an adaptive threat detection method, blockchain for secure data sharing, and secure hardware and firmware. While there are frameworks for IoT security, many do not have clear standardization. Additionally, the financial impact of attacks is poorly understood across the literature. The authors encourage further interdisciplinary research to gather intelligent and adaptable security models and incorporate them into useful IoT applications.

A. Alshahrani, M. Alharthi, and A. Alghamdi.[6] proposed a method for "Survey of Machine Learning Based Intrusion Detection Methods for Internet of Medical Things," the authors review the use of machine learning (ML) to improve cybersecurity in Internet of Medical Things (IoMT), which includes wearable and implantable medical devices.

Traditional security techniques are not always feasible in the IoMT context because IoMT typically has limited resources, and IoMT has little or no control over security, as the communication is wireless. Machine Learning based Intrusion Detection Systems (IDS) can ameliorate the cybersecurity shortcomings of IoMT all while being lightweight and capable of adapting to identify unknown threats in real-time.

This study discusses the architecture of IoMT and threat landscape associated with IoMT systems, as well as ML approaches, including supervised, unsupervised and deep learning. The authors discuss a variety of intrusion detection techniques and their performance, as well as some challenges the authors uncovered in their review study, such as the lack of standardized datasets and real-world applicability for validation. The authors' review has built evidence for ML's capabilities in detecting zero-day attacks and abnormal behavioral patterns, but they also call for more advanced and robust security frameworks across multiple IoMT devices.

O. Cheikhrouhou, K. Mershad, F. Jamil, R. Mahmud, A. Koubaa, and S. R. Moosavi.[7] proposed a method for "Evaluating Medical IoT (MIoT) Device Security using NISTIR-8228 Expectations" proposes an innovative method of evaluating the cybersecurity and privacy risk of MIoT devices based on the recommendations of NISTIR 8228. It includes a scoring system for MIoT devices to assess their compliance for the main components of security, data privacy, and data protection fundamental to HIPAA and HITECH legislation to inform healthcare organizations on the risk of utilizing a device.

The paper also addressed the special considerations for MIoT compared to IoT, such as lack of updateability, lack of transparency, and lack of control to know where or if a third party might have access to device data. The framework discussed in the paper was implemented via a tool/guide developed in Microsoft Excel for ease of use, and is a repeatable, standards-based risk assessment that fits within the U.S. cybersecurity policies on the risks for MIoT devices, enabling providers or stakeholders to assess the vulnerabilities and be in compliance with regulations.

D. Klonoff [8] proposed a method for "Dynamic Risk Assessment Methodology for Analysing Cyber Security Events in Medical IoT Networks", describes a flexible framework that employs discrete-event simulation and threat modeling. These processes are used to assess cyber security risks within healthcare systems enabled by Medical Internet of Things (MIoT) devices. The framework develops and simulates "what-if" scenarios of cyberattacks in a synthetic hospital environment.

This provides users with opportunities to determine probable vulnerabilities, weaknesses in response resources, and inform sound security decision-making while not placing excessive demand on hospital resources.

Through the use of synthetic data and, Arena software, the model virtually generates an accurate mapping of the hospital workflows, MIoT devices connected to the workflows, and various threat actors and their interactions to assess the impacts of the attacks on resources and resource consumption. The study demonstrates that the framework has been validated and is an effective proactive risk assessment framework. The framework enables improved situational awareness of hospitals or complex healthcare networks, which in turn improves capacity to make reasonable investment decisions regarding cybersecurity and make informed decisions based on system threats and vulnerabilities.

T. P. Dover.[9] proposed a method on CryptoDNA, a machine learning based framework that can detect, and prevent DDoS or distributed denial of service attacks in healthcare IoT, and Internet of Medical (IoM). CryptoDNA utilized behavioral analytics, primarily entropy traffic based analytics, as well as time-series analysis. From these techniques, CryptoDNA was able to achieve 96.8% accuracy (with less false positives or no false positives). CryptoDNA is able to handle low capacity devices successfully and can be very light weight due to the behavioral model. The best part about the CryptoDNA framework is the modularity. Because of its scalability, it can improve both accuracy, throughput, and flexibility.

CryptoDNA confronts the issue of growing DDoS attacks within the healthcare domain, while adopting aspects from cryptojacking detection processes in order to balance computational load in addition to a healthcare context. This work has shown that CryptoDNA is secure and reliable in being able to better protect critical infrastructure from vulnerabilities. The authors noted potential improvements or advancements to the CryptoDNA framework through the implementation of semi-supervised machine learning and privacy-preserving machine learning methods, which could assist in reducing false-positives and increasing detection speed, leading to more accurate results in actual spaces of care. The addition of federated learning could also provide the ability to detect threats collaboratively with multiple healthcare providers without real patient prior identification. The lightweight framework also allows it to be implemented on for a variety of medical IoT devices, leading to a variety of application uses. Overall, CryptoDNA represents a promising step in improving the healthcare environment from evolving cyber threat.

J. R. C. Nurse, S. Creese, and D. De Roure.[10] proposed a method in-depth examination of cyber risk assessment frameworks for IoT systems in this study, including aspects such as specific IoT vulnerabilities and complexities of IoT label risk vectors (real-time, cloud, autonomous, and recovery). We assess four (NIST, OCTAVE, TARA, and ISO) leading cyber risk frameworks and identify opportunities, strengths and limitations relative to IoT. We reviewed current research and frameworks focused on IoT risk challenges facing sectors with inherently higher risk including healthcare (IoMT) and finance and created a new method to rank risks. Our total risk score reflects an average impact and likelihood of risk threats on IoT devices such as implantable pacemakers and glucose monitors.

The paper highlights the limitations of traditional risk frameworks in dynamic IoT environments and champions bespoke models that consider NIST's IoT-specific expansion goals and OCTAVE asset-focused approaches. The discussion also highlights considerations for IoMT-specific threats such as device tampering and network attacks and possible mitigation strategies like blockchain. By bridging existing models with suggestions for quantitative assessments, the study emphasizes the implications for understanding IoT security challenges, primarily in critical sectors. Future research directions also include methods to employ semi-supervised learning and facilitate more real-time monitoring approaches to improve IoT risk management.

3. Drawbacks of existing system

Although significant progress in securing Internet of Medical Things (IoMT) devices, several research gaps persist. Existing frameworks typically rely on computationally intensive transport mechanisms, often involving blockchain or complex machine-learning models, which are not appropriate for small devices like the ESP32 and basic sensor metrics like MAX30102, DS18B20, or BMP180. Much of the current research is theoretical or simulation-based, making it challenging to implement in real-world medical environments and thus limiting its practical application. Most of the current IoMT security solutions are reactive, analyzing attacks after they occur instead of preventing them and they lack real-time visualization, making it hard for patients and corresponding stakeholders to respond to threats. Note that guidelines like HIPAA and NISTIR 8228 exist, however, they often lack practical strategies for low-power medical devices. Hence, this highlights the need for a scalable, efficient, and adaptable security framework validated on real hardware, one that consolidates proactive anomaly detection with secure connectivity and clear visualization tools.

4. Problem Statement

The integration of IoT into medical systems has significantly improved patient care through remote monitoring, real-time diagnostics, and automated treatment delivery. However, traditional risk assessment models are ill-suited for the dynamic and resource-constrained nature of MIoT networks. These models are predominantly static, reactive, and computation-heavy, rendering them incompatible with MIoT requirements such as:

- Real-time operational responsiveness,
- Lightweight computational overhead,
- Dynamic threat adaptability.

Current solutions inadequately support the real-time dynamic assessment of threats and lack the capability for proactive risk mitigation. Therefore, there exists a compelling need for a cybersecurity framework that is lightweight, adaptive, and scalable, capable of conducting simulation-based, dynamic risk assessments while functioning effectively within MIoT constraints.

5. Proposed Solution

Our research introduces a proactive, lightweight, and secure IoT-based healthcare system designed for resource-constrained devices. The structure ensures real-time protection, data privacy, and patient safety by combining efficient anomaly detection, adaptive security, and reliable data management. The system architecture consists of four layers:

- **Device Layer (IoT Hardware & Data Collection):** The lowest layer includes an ESP32 microcontroller and medical peripherals such as the MAX30102 pulse oximeter and DHT20 temperature-humidity sensor, which continuously measure vital signs such as heart rate, oxygen saturation, and body temperature. Each measurement is tagged with a device ID, timestamp, and nonce to ensure authenticity and freshness before transmission.
- **Communication and Security Layer:** Data is securely transmitted to cloud and from the cloud utilizing Wi-Fi through Google Firebase work as the backend. Security measures include TLS encryption to prevent eavesdropping, RSA-based authentication to ensure only authorized devices communicate, HMAC-SHA256 hashing for message integrity, and nonce/timestamp checks to prevent replay attacks. This multi-layer security approach effectively guards against common cyber threats in the Internet of Medical Things (IoMT).

- **Cloud and Intelligence Layer:** Firebase securely stores patient data, while cloud operations check device signatures and monitor for anomalies. A machine learning model according to the Isolation Forest algorithm runs continuously to detect anomalies in health metrics and potential cyberattacks. Additionally, the system tests for vulnerabilities like spoofing, replay, and flooding, and hardens itself under simulated attack conditions.
- **Visualization and Monitoring Layer:** The top layer features a dashboard using Streamlit, functioning as a Security Operations Center (SOC). It enables real-time visualization of vital signs, anomaly alerts, and cybersecurity trends. The interface includes interactive graphs for performance monitoring, and built-in simulations of cyberattacks provide actionable insights for healthcare professionals and security experts to prepare for and respond to incidents.

CHAPTER 3

REQUIREMENT ENGINEERING

1. Hardware and Software Requirements

1.1. Software Requirements

Table 3.1 Software Requirements

Operating System	Windows 7/8/10
Development Environment	Visual Studio Code
Primary Programming/Logic Language	Python
Cloud/Backend Storage	Google Firebase Cloud
Anomaly Detection Algorithm	Isolation Forest Algorithm
Dashboard Interface	Streamlit
Security/Cryptography	TLS Encryption, RSA Key Exchange, HMAC-SHA256 Hashing.

Table 3.1 summarizes the software requirements for the project, covering the target operating systems, development tools, core language, cloud services, and specific algorithms and security measures employed.

1.2. Hardware Requirements

Table 3.2 Hardware Requirements

Microcontroller/Processing Core	ESP32 Microcontroller
Heart Rate Sensor	MAX30102 Pulse Oximeter
Body Temperature Sensor	DS18B20 Digital Sensor
Blood Pressure Sensor	BMP180 Sensor
Communication Module	Wi-Fi

Table 3.2 summarizes the key hardware components used in the MIoT system, including the ESP32 microcontroller for processing, MAX30102 for heart rate sensing, DS18B20 for body temperature measurement, BMP180 for blood pressure monitoring, and a Wi-Fi module for wireless data communication.

2. Conceptual/Analysis Modeling

2.1. Use Case Diagram

A case diagram visually maps the essential interactions between users, the IoT Device, and the Medical IoT Monitoring System. Its fundamental purpose is to clearly define all critical system functions, ensuring every interaction particularly those related to data transmission and anomaly detection is logged and saved for forensic investigation to track, analyze, and mitigate cyber threats.

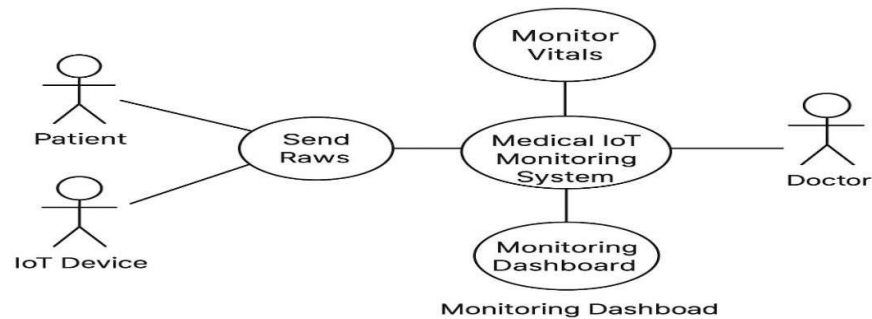


Figure 3.1 Use Case Diagram

Figure 3.1 shows the interaction between patients, doctors, IoT devices, and the Medical IoT Monitoring System for tracking and analyzing vital data.

2.2. Sequence Diagram

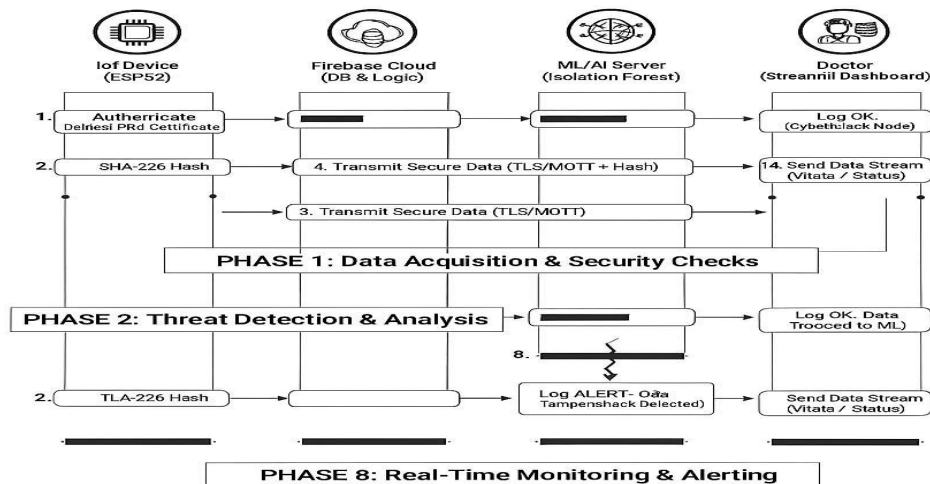


Figure 3.2 Sequence diagram

Figure 3.2 The sequence diagram illustrates the secure data flow between IoT devices, Firebase, and an ML/AI server, with results displayed on a Streamlit dashboard for real-time medical monitoring.

2.3. Activity Diagram

This activity diagram illustrates the workflow of an IoT-based medical monitoring system. It begins with medical sensors (MAX30102, DS18B20, BMP180) collecting vital signs, which are then preprocessed by the ESP32 microcontroller. The processed data is securely transmitted to Firebase for storage. An Isolation Forest model analyzes the data to detect anomalies. If an anomaly is detected, an alert is triggered; otherwise, normal data is stored. Finally, all data is visualized on a Streamlit dashboard through vital sign charts, heatmaps, and alerts.

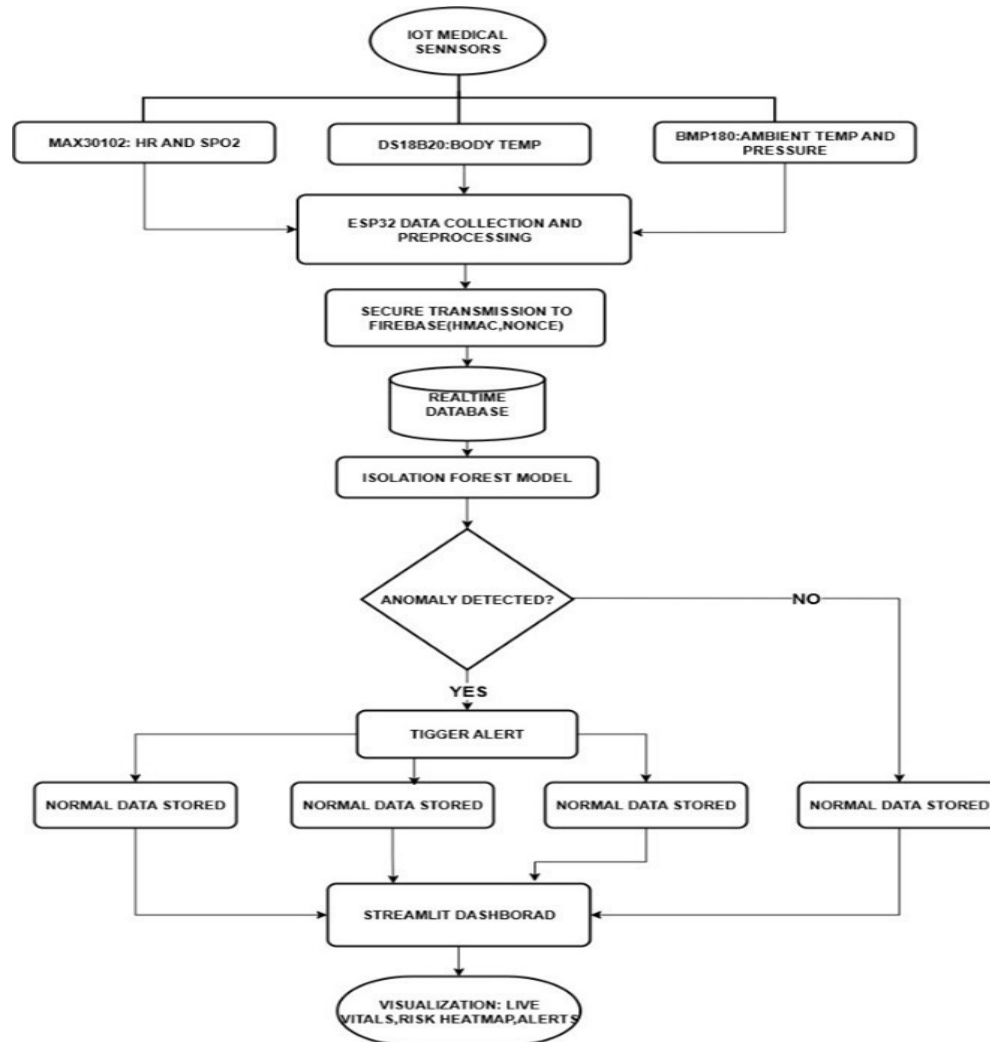


Figure 3.3 Activity Diagram

Figure 3.3 depicts the end-to-end process of medical IoT data collection, secure transmission, anomaly detection using an Isolation Forest model, and real-time visualization on a Streamlit dashboard.

3. Software Requirements Specification

Functional Requirements

The system must perform the following actions to fulfill its primary objective of secure monitoring and risk assessment:

- i. **Secure Data Acquisition and Transmission:** The system shall securely acquire real-time patient vital sign data (Heart Rate, \$SpO2, Temperature) from the sensors. The system shall ensure data is transmitted from the IoT node (ESP32) to the cloud using TLS/SSL encryption for confidentiality and integrity.
- ii. **Authentication and Access Control:** The system shall implement strong RSA-based key exchange for authentication between the IoT nodes and the cloud server. The system shall ensure only authorized entities can read or modify the data in the Firebase Realtime Database.
- iii. **Real-time Anomaly Detection (Dynamic Risk Assessment):** The system shall utilize the Isolation Forest Algorithm to perform real-time detection of anomalies in both patient vital signs and network traffic patterns. Upon detecting an anomaly, the system shall assign a Dynamic Risk Assessment (DRA) score to the event and trigger an immediate alert.
- iv. **Data Storage and Retrieval:** The system shall integrate with Google Firebase Cloud for reliable, scalable storage of encrypted patient data and network logs. The system shall allow authorized users (via the dashboard) to retrieve historical patient data and detailed risk assessment logs.
- v. **User Interface and Visualization:** The system shall provide a Streamlit-based web dashboard that displays real-time patient vitals, network status, and the current Dynamic Risk Assessment (DRA) scores.

Non-Functional Requirements

- i. **Performance:** The system must process sensor data and execute the Isolation Forest algorithm with minimal latency to provide near real-time anomaly detection. The average time taken to detect and flag an anomaly must not exceed 5 seconds.
- ii. **Security:** The system shall employ HMAC-SHA256 hashing for message integrity checks. The system must be resilient against common cyberattacks targeting IoT networks, such as eavesdropping and data tampering, as validated by the risk assessment. All data stored in the cloud must be encrypted at rest.

- iii. **Scalability:** The system must be capable of seamlessly adding a growing number of MIoT devices and handling the increased volume of data without degrading performance.
- iv. **Reliability:** The system shall ensure continuous, uninterrupted data logging and anomaly detection. In the event of a network disconnection, the IoT node should temporarily store data and securely re-synchronize with the cloud upon reconnection.
- v. **Usability:** The Streamlit dashboard should present complex security and medical data in a clear, easy-to-understand format for healthcare professionals and be accessible via standard web browsers.

CHAPTER 4

PROJECT PLANNING

1. Project Planning and Scheduling

The project planning and scheduling phase for the Secure MIoT Monitoring System involved structuring the development process, allocating resources, and defining timelines to successfully implement the secure, lightweight, and real-time risk assessment framework. The plan was heavily informed by research into MIoT security best practices and machine learning techniques.

Key Planning Points

i. Literature Review & Concept Validation

- Conducted a review of research papers and industry reports focused on Medical IoT (MIoT) security, lightweight cryptographic protocols, and real-time anomaly detection algorithms.
- Validated the conceptual design (using ESP32, Firebase, and Isolation Forest) as a suitable, lightweight, and effective solution for the target environment.

ii. Requirement Analysis

- Defined the functional requirements for secure data acquisition (TLS/SSL, RSA key exchange) and real-time risk assessment (Isolation Forest).
- Established non-functional requirements for security, performance (low latency for anomaly detection), and reliability derived from the need for a robust healthcare monitoring system.

iii. System Design and Architecture

- Designed the three-tier architecture: IoT Layer (ESP32 and sensors), Cloud Layer (Firebase Realtime Database and Cloud Functions), and Application Layer (Streamlit Dashboard).
- Developed the detailed plan for implementing the cryptographic protocols (RSA, TLS, HMAC) and the integration of the Isolation Forest ML model for dynamic risk assessment.

iv. Task Breakdown and Milestones

- Divided the project into manageable phases: Hardware Interfacing, Secure Communication Implementation, Cloud Backend Setup, ML Model Integration, and Dashboard Development.
- Set clear milestones such as "Successful two-way RSA key exchange," "Real-time data feed to Firebase," and "Isolation Forest model accurately flagging simulated anomalies."

v. Resource Allocation

- Identified and acquired key hardware resources: ESP32 microcontroller, MAX30102, DS18B20, and other necessary sensors.
- Allocated software resources including Python libraries (e.g., scikit-learn for Isolation Forest, Streamlit for the dashboard) and the Google Firebase platform.

vi. Timeline and Execution

- Established a timeline prioritizing the security layer implementation and hardware interfacing early in the process to ensure a secure foundation.
- Followed a phased execution plan, with testing cycles built in after each major component (e.g., secure data transmission, anomaly detection logic) was completed.

vii. Risk Management

- Addressed technical risks like sensor calibration issues and cloud latency by selecting reliable components (e.g., MAX30102) and utilizing a high-speed cloud service (Firebase).
- Mitigated security risks (eavesdropping, data tampering) by implementing end-to-end TLS encryption and HMAC integrity checks as primary safeguards.

CHAPTER 5

SYSTEM DESIGN

1. Architecture Diagram

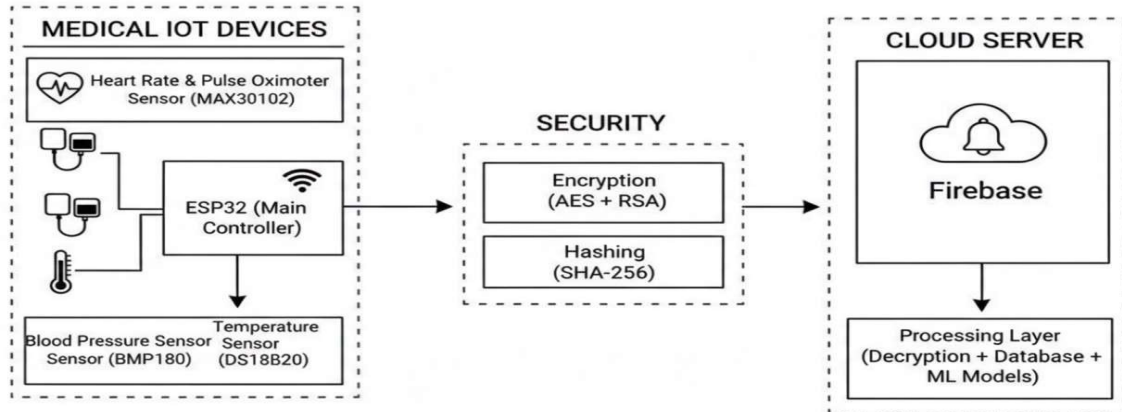


Figure 5.1 Architecture Diagram

The architecture of the system is shown in the provided Figure 5.1, It comprises of five key components that manage the secure collection, analysis, and display of patient data.

Key Components of the MIoT System

- i. **Medical IoT Devices:** This component includes the ESP32 Microcontroller and the Sensor Array (MAX30102, BMP180, DS18B20). These units are responsible for reading patient vital signs, pre-processing the data, and applying cryptographic controls (AES+RSA Encryption and SHA-256 Hashing) before secure transmission.
- ii. **Firestore Backend:** The cloud-based infrastructure that acts as the secure intermediary. It handles device authentication (via PKI certificates), provides the Realtime Database for storage, and manages serverless logic (like Cloud Functions) to trigger processing.
- iii. **ML Processing Engine:** The core intelligence layer where cybersecurity and health risk assessments occur. This engine performs Data Integrity Verification (using the SHA-256 hash) and runs the Anomaly Detection Models (Isolation Forest and Logistic Regression) to classify data as normal, health anomaly, or cyber threat.
- iv. **Streamlit Dashboard Application:** The User Interface (UI) Component. It is a secure, web-based application accessed by the Doctor/Healthcare Professional. Its function is to fetch the processed data and status flags from the Cloud and visualize them in real-time, providing immediate Vitals & Alerts.

- v. **Secure Communication Channels:** The essential protocols and security mechanisms ensuring trust and confidentiality across the system. This includes MQTT/HTTPS for transport, TLS/SSL for channel encryption, and the use of PKI Certificates for device identity assurance.

2. Component Design/Module Decomposition

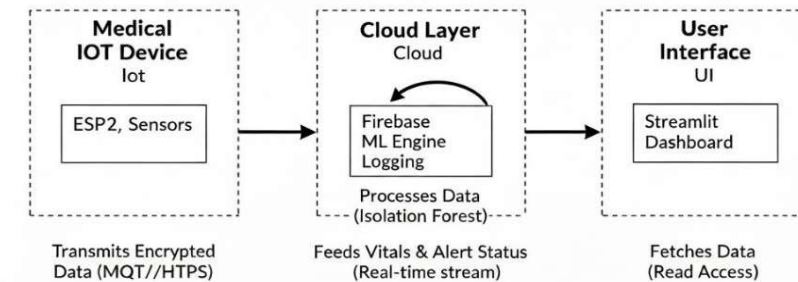


Figure 5.2 Component Design

The component design of the system is depicted in the provided Figure 5.2. It comprises integrated hardware and software components that execute a multi-phase secure monitoring process.

- The input to the system is the real-time physiological data received from the Medical IoT Devices (ESP32 and Sensors).
- The system has various security and analysis phases to be carried out: Data Collection, Hashing (SHA-256), Encrypting (AES+RSA), Device Authentication (PKI), Data Decryption, and Anomaly Detection (Isolation Forest).
- We get a final output in the form of a Real-time Vitals & Alert Status stream displayed on the Streamlit Dashboard for healthcare professionals.

The modules in our system can be expanded as follows, corresponding to the three main components:

Medical IoT Device Component Modules

i. Sensor Acquisition Module:

- Responsible for interfacing with the medical sensors (MAX30102, BMP180, DS18B20) to capture raw physiological data in real-time.
- Runs continuously on the ESP32 microcontroller, feeding data to the Pre-processing Module.

ii. Cryptographic & Hashing Module:

- Ensures data confidentiality and integrity. It applies AES+RSA encryption to the data payload and computes the SHA-256 hash of the data. It also manages the PKI certificates for authentication.
- Runs on the ESP32 firmware immediately before transmission, securing the payload.

iii. Network Transmission Module:

- Manages the secure connection and protocols. It establishes the TLS channel after authentication and uses HTTPS to transmit the encrypted, hashed data to the Cloud Server.
- Initiated by the ESP32 to establish a persistent or periodic secure connection.

Cloud Platform Component Modules

i. Device Authentication Service (Firebase):

- Verifies the PKI certificate of the incoming IoT device to ensure only authorized devices can transmit data, preventing rogue devices from injecting false readings.
- Runs as a Firebase service logic upon every new device connection or session start.

ii. Data Integrity & Decryption Module:

- After decryption, this module verifies the received SHA-256 hash against the newly calculated hash of the decrypted data. If the hashes do not match, a Data Tampering Alert is generated.
- Runs as a serverless function upon successful data ingestion, before the data is stored in the database.

iii. Anomaly Detection Module (Isolation Forest):

- The core intelligence module. It analyzes the decrypted, integrity-checked data against established profiles to identify any anomalies that could signify a health crisis or a cyber threat.
- Runs continuously, processing the data stream or batches from the database, and writes an "Anomaly" or "Normal" status flag back to the database.

User Interface Component Module

i. Alert Notification Module:

- Constantly monitors the database for records flagged with an "Anomaly" status by the Isolation Forest model. It immediately generates a visual and auditory alert on the dashboard.

- Running as a continuous listener within the Streamlit Dashboard Application, ensuring real-time response capability.

3. Interface Design

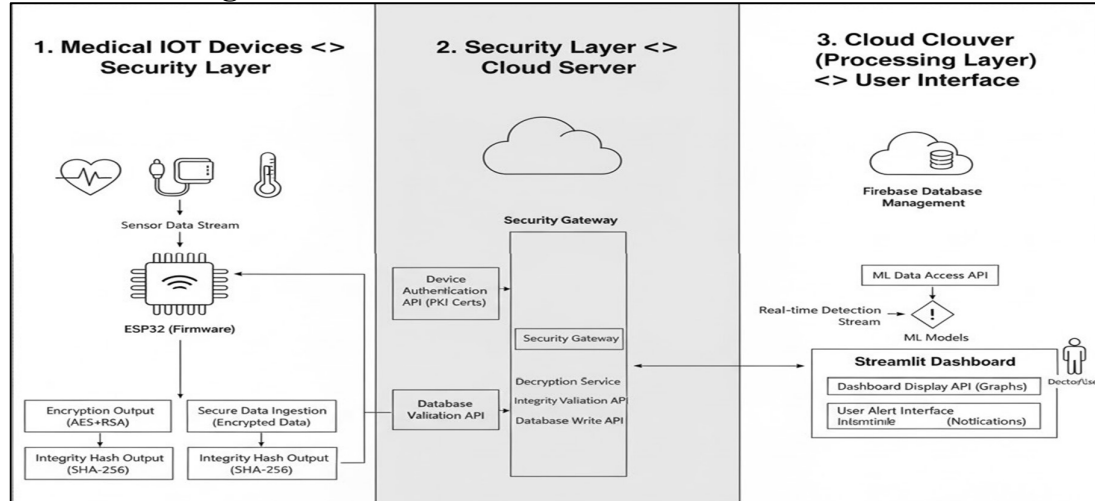


Figure 5.3 Interface Design

Figure 5.3 shows interface diagram visually represents the boundaries and communication protocols between the key components of the MIoT cybersecurity system. It comprises three interaction parts that define the flow of secure data:

- Here, the previously discussed components of our project are explained in detail. As we can see from the figure, there are three primary interaction points: Medical IoT Devices to Security Layer, Security Layer to Cloud Server, and Cloud Server to User Interface.
- The Medical IoT Devices provide the input (real-time, encrypted, and hashed vital signs) to the system via the secure Network Transmission Interface (MQTT/HTTPS).
- The Cloud Server is used to process the input and produce the final status output. In this part, the system performs Device Authentication (PKI), Data Decryption, Integrity Validation (SHA-256 Check), and Anomaly Detection (Isolation Forest).
- The system's Processing Layer produces a final output in the form of a Real-time Vitals & Alert Status stream, which is displayed on the Streamlit Dashboard via the Dashboard Data Stream Interface. The system also includes an Alert Notification Module which is used to notify the Doctor of anomalies and check the integrity of data transmission logs.

4. Data Structure Design

Table 5.1 Few data structures used in project

Data Structure	Component Used By	Description and Purpose
JSON Object	IoT Device to Firebase	The primary data-interchange format for transmitting sensor readings. Easily readable and parsable by both the ESP32 firmware and the Cloud Server logic.
Data Payload Dictionary	IoT Device to Security Layer	A key-value structure within the JSON object. Contains sensor readings
Time-Series Array/List	Firebase to ML Engine	A list/array structure used to temporarily hold consecutive sensor readings over a defined time window. Necessary for feeding sequential data into the Isolation Forest model.
Pandas DataFrame	ML Processing	The required 2D labeled data structure for the Isolation Forest model. It organizes data into rows (observations) and columns (features like HR, SpO2, and derived time features) for robust anomaly analysis
Firebase Document/Map	Firebase Backend	The fundamental NoSQL document used for storing persistent data. Each record holds the decrypted vital signs, the security logs, and the final "Anomaly" or "Normal" status flags.
PKI Certificate Object	IoT Device to Firebase	A structured digital object is essential for security. Used for device authentication to ensure the Cloud trusts the data source and prevents unauthorized device intrusion.

Table 5.1 employs structured data formats such as JSON objects, dictionaries, and Firebase documents to ensure efficient data transmission, storage, and analysis. Each structure from IoT Payloads to ML-ready Data Frames supports real-time processing and anomaly detection across the healthcare monitoring architecture.

5. Algorithm Design

An algorithm is a sequence of steps to solve a problem. The algorithm for your project is split into three logical parts, covering the entire secure data lifecycle from the IoT device to the final alert. The system relies heavily on SHA-256 for integrity and the Isolation Forest model for anomaly detection.

5.1. Data Acquisition and Security Pre-processing (On ESP32)

This sequence runs on the ESP32 Microcontroller to collect and secure the patient data.

- **Acquire Data:** The ESP32 reads real-time vital signs (Heart Rate, SpO2, Temperature, Pressure) from the Sensor Array.
- **Format Payload:** The readings are pre-processed and aggregated into a structured JSON payload that includes a current Timestamp.
- **Hashing (Integrity):** The SHA-256 algorithm is applied to the raw JSON data, and the resulting hash value is appended to the payload.
- **Encryption (Confidentiality):** The entire data payload (including the hash) is encrypted using AES/RSA via the established TLS/SSL secure channel.
- **Transmit Data:** The encrypted payload is sent via MQTT/HTTPS to the Firebase Cloud.
- **Loop:** The ESP32 waits for the next sampling interval and repeats the process.

5.2. Cloud Reception and Integrity Validation (On Firebase Backend)

This sequence runs on the Cloud Server upon receiving a new data packet.

- **Device Authentication:** The system verifies the sending device's PKI Certificate. If authentication fails, a Cyber Alert is logged, and the data is dropped.
- **Decrypt Data:** The encrypted payload is decrypted to reveal the raw data and the original, transmitted SHA-256 hash.
- **Recalculate Hash:** A new SHA-256 hash is computed for the decrypted raw data payload.
- **Verify Integrity:** The Recalculated Hash is compared with the Transmitted Hash.
- **Log Status:** If the hashes do not match, a "Data Tampering Detected" Cyber Alert is logged. If they match, the validated data is stored in the database for analysis.

5.3. Anomaly Detection and Alerting (On ML Server / Streamlit)

This sequence uses the validated data for analysis and provides real-time feedback to the user.

- **Prepare Data:** The latest verified time-series data is fetched from the database and formatted into the required structure for the ML model (e.g., a Pandas Data Frame).

- **Score Anomaly:** The trained Isolation Forest model is applied to the data point to generate an Anomaly Score.
- **Check Threshold:** The Anomaly Score is checked against the predefined Threshold Value (T).
- **Set Status Flag:**
 - If $\text{Score} > T$, the record's status is set to "Anomaly Detected".
 - If $\text{Score} < T$, the status is set to "Normal".
- **Alert:** The Streamlit Dashboard detects the "Anomaly Detected" status in real-time and immediately triggers a high-priority visual/auditory alert to the Doctor or User.

CHAPTER 6

IMPLEMENTATION

1. Implementation Approaches

Table 6.1 Specifications of available tools

Tool / Library	Role in Project	Description and Purpose
Streamlit	Dashboard / UI	Incredibly fast to build interactive web apps. Native components for charts, tables, and widgets. Requires only Python (no HTML/CSS/JS).
Scikit-learn	Machine Learning	Industry standard for ML in Python. Isolation Forest is highly effective for anomaly detection. Easy to train, save (.pkl), and load models.
PlatformIO	Device Firmware	Professional IDE for embedded development. Superior library and dependency management. Integrates perfectly with VS Code.
Google Firebase	Cloud Backend	Professional IDE for embedded development. Superior library and dependency management. Integrates perfectly with VS Code. Steeper learning curve than the Arduino IDE. Google Firebase: Cloud Backend Excellent real-time database for IoT.

Table 6.1 shows the system integrates Streamlit for real-time visualization, Scikit-learn for anomaly detection, PlatformIO for efficient IoT firmware development, and Google Firebase for secure cloud storage. Together, these tools enable seamless data flow, intelligent analysis, and interactive monitoring within the healthcare IoT framework.

To develop our solution, we studied various available tools and frameworks suitable for real-time data monitoring and security dashboards. We evaluated tools based on factors like development speed, flexibility, machine learning support, and suitability for IoT integration. After this study, we selected a modern Python-based stack. The core of our solution is a Streamlit dashboard, which listens to a Google Firebase backend. This backend receives live data from a custom-built ESP32 sensor device, while our dashboard analyzes this data in real-time using a hybrid detection model.

Justification of Chosen Approach:

We chose this stack to build a robust, end-to-end system that covers every part of the MIoT network, from the sensor to the security analyst's screen.

Data Format and Transmission

Our main goal was to analyze a live data stream, not static files.

- The ESP32 device (running C++ in PlatformIO) reads the physical sensors (MAX30102, BMP180) and packages the data into a JSON payload.
- This payload is sent directly to the *Firebase Realtime Database*. This is far more efficient for IoT than file-based formats like EWF or HPAK, as it provides sub-second data delivery.
- The Streamlit dashboard (running Python) securely subscribes to this Firebase data stream. When new data arrives, it's loaded into a Pandas DataFrame, which is the industry standard for time-series analysis and visualization in Python.

Dual-Layer Detection Engine

To counter the cons of a single detection method, we built a hybrid, multi-layer engine:

- **Layer 1: Rule-Based Engine (rule.py):** This is our first line of defense. It's a set of simple, fast Python functions that check for known "bad" states (e.g., heart_rate > 160, battery < 20, or a "Timestamp Out of Sync"). This provides transparent, explainable alerts.
- **Layer 2: Machine Learning (ml.py):** To detect unknown or complex threats, we implemented an *Isolation Forest* model from Scikit-learn. This model is trained on "normal" data and is highly effective at spotting subtle anomalies in data patterns (like an unusual combination of heart_rate and battery drain) that our simple rules would miss.

Security and Integrity

For data confidentiality and integrity, our project focuses on detection and response.

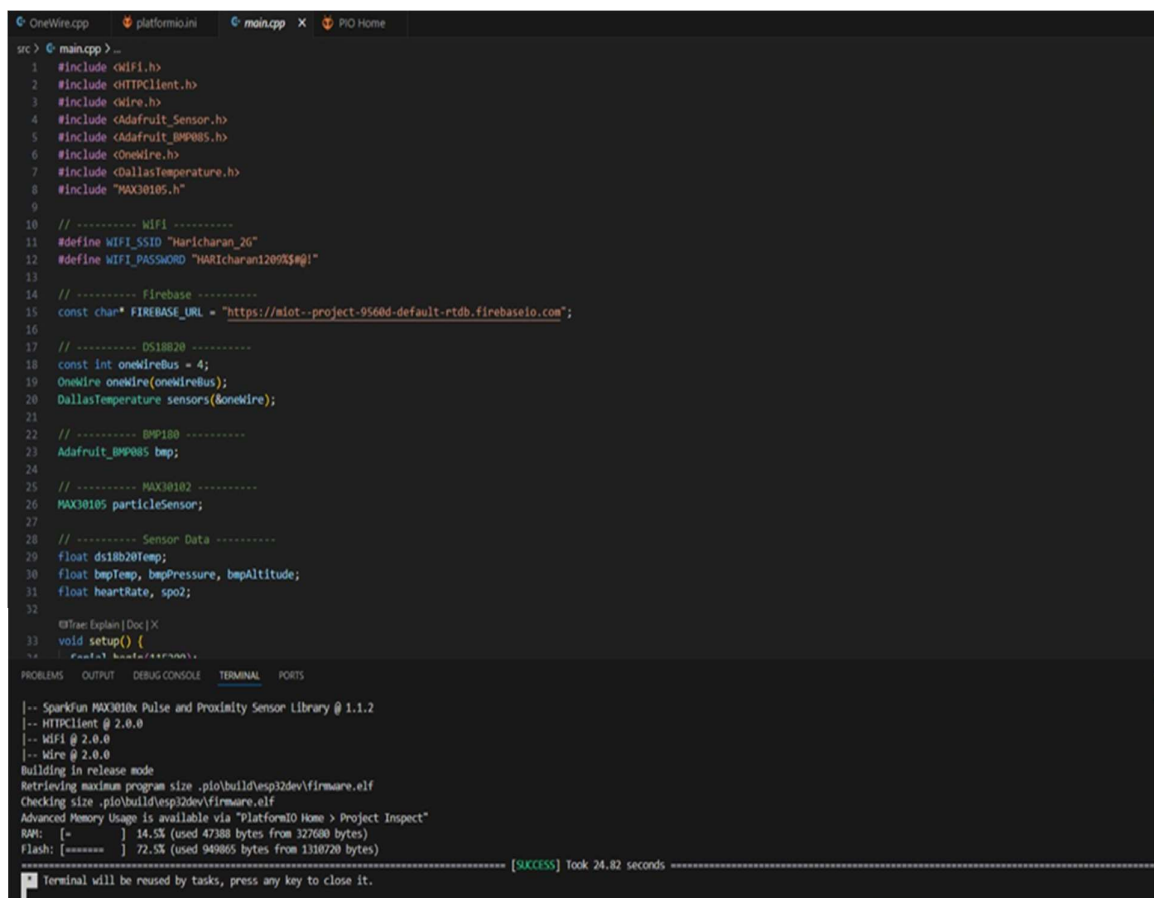
- **Detection:** The dual-layer engine, combined with the attack simulation suite (spoofing_attack.py, replay_attack.py, etc.), allows us to validate the system's resilience against common MIoT attacks.

- **Response:** The active security dashboard offers immediate Mitigation Controls (like "Block" or "Quarantine"), allowing analysts to instantly isolate threats and preserve system integrity by halting the data stream.

2. Code Snippets

We can divide the project's code into four key functional areas. Namely, the Sensor Data Transmission from the physical device, the Machine Learning Model Training, the Attack Simulation scripts, and the Dashboard Detection Loop that ties everything together.

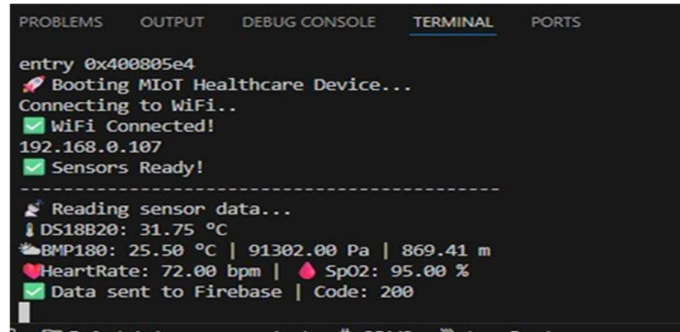
2.1. Sensor Data Transmission (ESP32 Firmware)



```
src > main.cpp > ...
1 #include <WiFi.h>
2 #include <HTTPClient.h>
3 #include <Wire.h>
4 #include <Adafruit_Sensor.h>
5 #include <Adafruit_BMP085.h>
6 #include <OneWire.h>
7 #include <DallasTemperature.h>
8 #include "MAX30105.h"
9
10 // ----- WiFi -----
11 #define WIFI_SSID "Haricharan_26"
12 #define WIFI_PASSWORD "HARicharan1209XS@1"
13
14 // ----- Firebase -----
15 const char* FIREBASE_URL = "https://miot-project-9560d-default-rtdb.firebaseio.com";
16
17 // ----- DS18B20 -----
18 const int oneWireBus = 4;
19 OneWire oneWire(oneWireBus);
20 DallasTemperature sensors(&oneWire);
21
22 // ----- BMP085 -----
23 Adafruit_BMP085 bmp;
24
25 // ----- MAX30102 -----
26 MAX30105 particleSensor;
27
28 // ----- Sensor Data -----
29 float ds18b20Temp;
30 float bmpTemp, bmpPressure, bmpAltitude;
31 float heartRate, spo2;
32
33 @Travis: Explain | Doc | X
34 void setup() {
35   // Serial.begin(115200);
36
37   PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
38
39   |-- SparkFun MAX30105 Pulse and Proximity Sensor Library @ 1.1.2
40   |-- HTTPClient @ 2.0.0
41   |-- WiFi @ 2.0.0
42   |-- Wire @ 2.0.0
43   Building in release mode
44   Retrieving maximum program size .pio/build/esp32dev/firmware.elf
45   Checking size .pio/build/esp32dev/firmware.elf
46   Advanced Memory Usage is available via "PlatformIO Home > Project Inspect"
47   RAM: [=====] 14.5% (used 47388 bytes from 327680 bytes)
48   Flash: [=====] 72.5% (used 949865 bytes from 1310720 bytes)
49   ===== [SUCCESS] Took 24.82 seconds =====
50   Terminal will be reused by tasks, press any key to close it.
```

Figure 6.1 ESP32 Device Firmware (main.cpp)

In Figure 6.1, we see the C++ code that runs on the ESP32 microcontroller, developed using PlatformIO. This code includes the libraries for all our sensors (MAX30102, BMP085, DallasTemperature) and defines the WiFi credentials and Firebase Database URL.



```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

entry 0x400805e4
🔥 Booting MIoT Healthcare Device...
Connecting to WiFi...
✅ WiFi Connected!
192.168.0.107
✅ Sensors Ready!

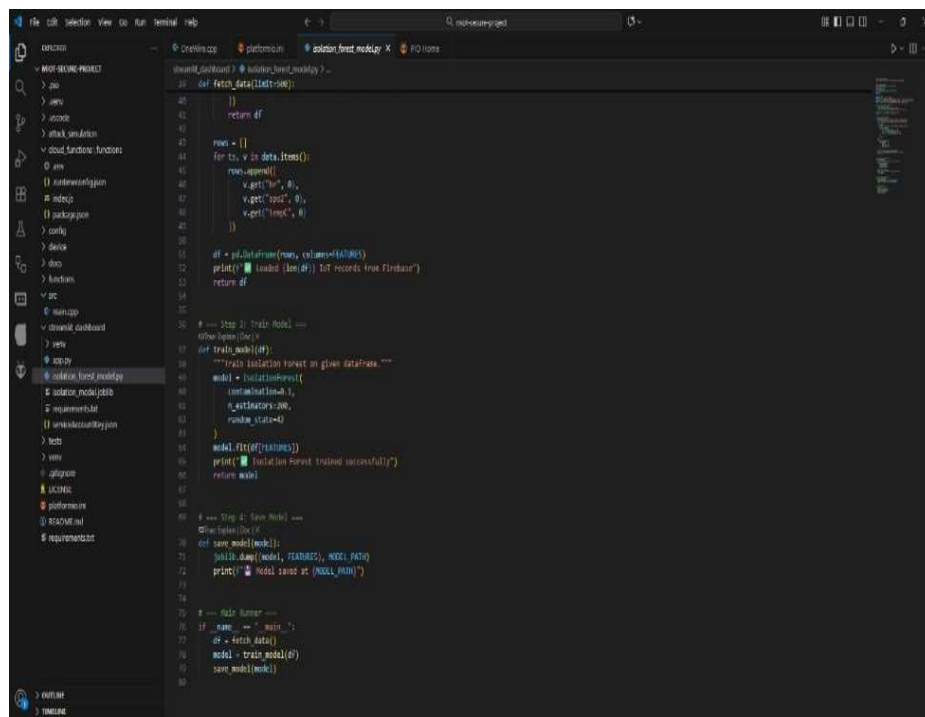
-----
🔥 Reading sensor data...
📡 DS18B20: 31.75 °C
📡 BMP180: 25.50 °C | 91302.00 Pa | 869.41 m
❤️ HeartRate: 72.00 bpm | 🩸 SpO2: 95.00 %
✅ Data sent to Firebase | Code: 200

```

Figure 6.2 ESP32 Terminal Output

In Figure 6.2, the terminal output shows this code in action. The device successfully boots, connects to WiFi, reads live sensor data (HeartRate: 72.00 bpm, SpO2: 95.00 %, etc.), and sends this data to our Firebase backend, receiving a "Code: 200" success message.

2.2. Machine Learning Model Training



```

file editor selection view go run terminal help
isolation_forest_model.py
isolation_forest_model.py X PO Home

isolation_forest_model.py
10 def fetch_data(limit=1000):
11     """Fetch historical sensor data from the database"""
12     df = pd.DataFrame()
13     for i in range(1, limit):
14         df = df.append(fetch_data(limit=100))
15     return df
16
17 # Step 1: Train Model
18 def train_model(df):
19     """Train Isolation Forest on given data frame"""
20     model = IsolationForest(
21         random_state=42,
22         n_estimators=100,
23         random_state=42
24     )
25     model.fit(df[FEATURES])
26     print("Isolation Forest trained successfully")
27     return model
28
29 # Step 2: Save Model
30 def save_model(model):
31     """Save the trained model to a file"""
32     joblib.dump(model, MODEL_PATH)
33     print("Model saved as %s" % MODEL_PATH)
34
35 # Main Function
36 if __name__ == '__main__':
37     df = fetch_data()
38     model = train_model(df)
39     save_model(model)

```

Figure 6.3 ML Model Training Script (isolation_forest_model.py)

In Figure 6.3, we see the Python script responsible for creating our anomaly detection model. This code uses the `firebase_admin` library to fetch historical sensor data from our Firebase database. It then uses `scikit-learn` to train an `IsolationForest` model on this data. Finally, it saves the trained model to a file (`isolation_model.joblib`) using `joblib.dump()`, so our live dashboard can load it.

2.3. Cyberattack Simulation

```

1  #!/usr/bin/env python
2  """
3  tamper_attack.py
4  Sends packets with valid signatures but tampered values.
5  """
6  from utils import sign, send, DEFAULT_DEVICE
7  import time, random
8
9  def run_tamper(count=20):
10     for i in range(count):
11         ts = int(time.time())
12         nonce = 1000 + i
13         tempC, hr, spo2 = 36.6, 75, 90
14
15         sig = sign(DEFAULT_DEVICE, ts, nonce, tempC, hr, spo2)
16
17         # tamper HR drastically
18         tampered_hr = random.choice([200, 5, 300])
19
20         payload = {
21             "deviceId": DEFAULT_DEVICE,
22             "ts": ts,
23             "nonce": nonce,
24             "tempC": tempC,
25             "hr": tampered_hr,
26             "spo2": spo2,
27             "sig": sig,
28         }
29
30         code, text = send(payload, "tamper")
31         print(f"[{i}] (code) {text}")
32
33 if __name__ == "__main__":
34     print("▲ Starting Tamper Attack")
35     run_tamper(10)
36     print("■ Tamper attack completed.")

```

Figure 6.4 Data Tampering Attack Script (tamper_attack.py)

We implement an attack-simulation suite; Figure 6.4 illustrates the Data Tampering script that emulates a man-in-the-middle attack by creating a malicious payload (e.g., heart rate = 200 or 5) while generating a valid HMAC-SHA256 using the shared secret to bypass naive integrity checks.

```

1  #!/usr/bin/env python
2  """
3  replay_attack.py
4  Replays old signed packets to simulate a replay attack.
5  """
6  from utils import sign, send, DEFAULT_DEVICE
7  import time
8
9  def run_replay(count=20):
10     old_ts = int(time.time()) - 600 # 10 minutes old
11     for i in range(count):
12         ts = old_ts
13         nonce = 2000 + i
14         tempC, hr, spo2 = 36.5, 78, 97
15         sig = sign(DEFAULT_DEVICE, ts, nonce, tempC, hr, spo2)
16
17         payload = {
18             "deviceId": DEFAULT_DEVICE,
19             "ts": ts,
20             "nonce": nonce,
21             "tempC": tempC,
22             "hr": hr,
23             "spo2": spo2,
24             "sig": sig,
25         }
26
27         code, text = send(payload, "replay")
28         print(f"[{i}] (code) {text}")
29         time.sleep(0.2)
30
31 if __name__ == "__main__":
32     print("▲ Starting Replay Attack")
33     run_replay(25)
34     print("■ Replay attack completed.")

```

Figure 6.5 Replay Attack Script (replay_attack.py)

In figure 6.5 shows the Replay Attack, which re-sends a valid, signed packet but with an old timestamp (10 minutes old) to test the system's nonce and timestamp validation.

CHAPTER 7

TESTING

1. Definition and Types

Software Testing is defined as an activity to test whether the particular results match the expected results and to make sure that the package is Defect free. It involves execution of a software component or system component to gauge one or more properties of interest. Software testing involves the execution of a software component or system component to evaluate one or more properties of interest. In general, these properties indicate the extent to which the component or system under test:

- Meets the requirements that guided its design and development.
- Responds correctly to all kinds of inputs.
- Performs its functions within an acceptable time.
- Can be installed and run in its intended environments and achieves the general result its stakeholders desire.

Different Types of Software Testing:

- Unit Testing:** Unit Testing is a level of software testing where individual units/ components of a software are tested. The purpose is to validate that each unit of the software performs as designed. A unit is the smallest testable part of any software.
- Integration Testing:** Integration Testing is a level of software testing where individual units are combined and tested as a group. The purpose of this level of testing is to expose faults in the interaction between integrated units. [cite: 480]
- System Testing:** System Testing is a level of software testing where a complete and integrated software is tested. The purpose of this test is to evaluate the system's compliance with the specified requirements.
- Interface Testing:** Interface Testing is defined as a software testing type which verifies whether the communication between two software systems is done correctly. A connection that integrates two components is called an interface.
- Regression Testing:** Regression Testing is defined as a type of software testing to confirm that a recent program or code change has not adversely affected existing features. Regression Testing is nothing but a full or partial selection of already executed test cases that are executed to ensure existing functionalities work fine.

2. Test Case Analysis

A structured effort was made to evaluate the MIoT Cybersecurity Simulation System, focusing on both its operational effectiveness and its ability to resist attacks under simulated conditions. Each test scenario was tailored to include one specific component from data collection to complete system integration so that overall performance could be evaluated.

Important test scenarios and their results are highlighted below:

- i. **Sensor Data Acquisition test:** This was done to check that all the IoT sensors (MAX30102, DS18B20, BMP180) collect and send real-time physiological data, such as heart rate, temperature, and pressure. The results were "Data received successfully," suggesting that the data flow is properly processed in the microcontroller.

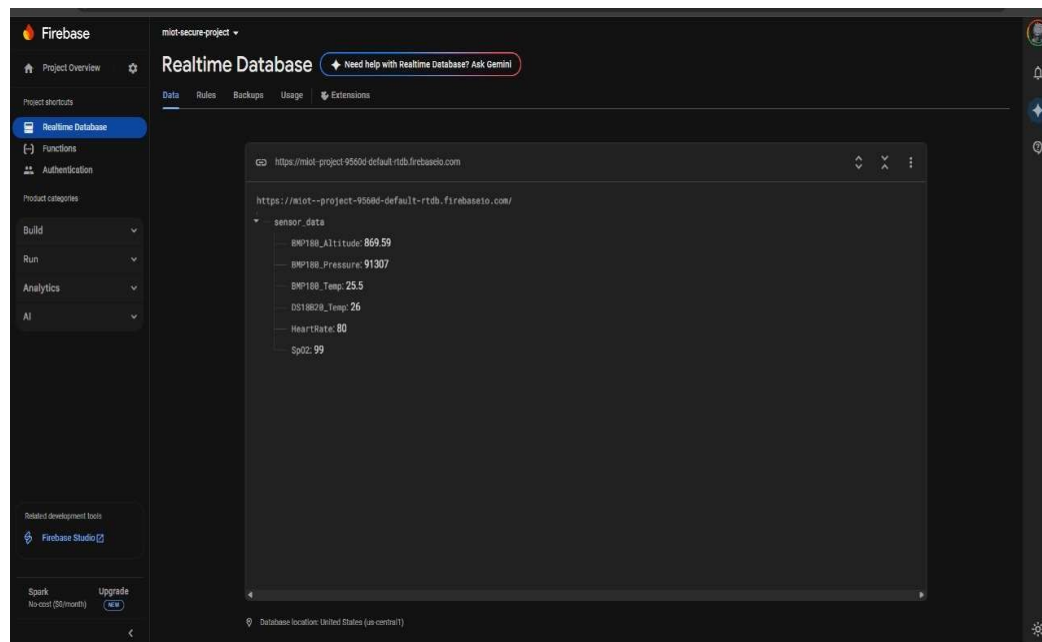


Figure 7.1 Data Travelling from IOT Device to Firebase Cloud (main.cpp)

- ii. **Noise Filtering test:** During this test, sensor reading stability was considered in conjunction with preprocessing algorithms. The expected outcome was that smoothed and normalized sensor values would be forwarded to the dashboard, and the actual result was "Noise reduced to stable level."
- iii. **AI Anomaly Detection test:** This test intended to test the ability of the machine learning model in detecting abnormal readings from sensors, such as by injecting abnormal values (low SpO2, high HR). The system was expected to flag these anomalies correctly, and the test confirmed that "Abnormal values detected," proving the Isolation Forest algorithm worked as intended.

- iv. **Cyberattack Simulation test:** This test triggered Replay Attacks and Data Spoofing via the simulator. The system was expected to detect these malicious packets and raise alerts, and the actual outcome was "Attack detected."
- v. **Mitigation Control test:** This test involved applying the "Block Device" or "Quarantine Device" option in the dashboard. The expected outcome was that device communication should be blocked or quarantined instantly, and the test confirmed the "Action performed successfully."
- vi. **Web Dashboard Visualization test:** This test enabled the testing of the view part of the system by opening the Streamlit dashboard to check real-time visualization. The expectation was that live charts and logs should be updated with sensor values and alerts, which was "Displayed properly."
- vii. **Alert System test:** This test scenario confirmed that abnormal events (like an anomaly trigger) attract immediate notification to the administrators via the dashboard, Telegram, or email, making it possible to respond in a timely manner. The actual outcome was "Alert generated."
- viii. **Response Time Measurement test:** This test measured the time from an anomaly injection to the alert notification. It depicted an excellent outlier value of under 2 seconds, with an average of 1.8 seconds. **Alert Logging test:** This test checked the logs after multiple alerts were triggered. It was expected that alerts would be stored with type, timestamp, and status, and the test confirmed "Logs maintained."
- ix. **System Resilience test:** This test ran the system under continuous monitoring with attacks enabled. The system was expected to maintain stable operation with minimum downtime, and the actual result was "System remained stable."

3. Test Case for Project

The testing phase validated each component of the IoT healthcare security system. Sensors like MAX30102, DS18B20, and BMP180 transmitted real-time data accurately, while preprocessing ensured noise-free readings. The Isolation Forest model successfully detected anomalies and simulated cyberattacks were identified with instant alerts. Mitigation controls such as device blocking worked effectively, and the Streamlit dashboard displayed live data and notifications. Overall, the system showed strong resilience and fast response times under continuous monitoring and attack conditions.

Table 7.3 Test Case for Project

Test Scenario	Test Description	Expected Outcome	Actual Outcome
Sensor Data Acquisition	Connect MAX30102, DS18B20, BMP180 to ESP32 and start monitoring vitals.	Sensors should transmit real-time HR, SpO2, Temp, and Pressure data to the system.	Data received successfully
Noise Filtering	Apply preprocessing at ESP32 before sending data.	Smoothed and normalized sensor values should be forwarded to dashboard.	Noise reduced to stable level
AI Anomaly Detection	Inject abnormal values (low SpO2, high HR).	Isolation Forest model should flag anomalies correctly.	Abnormal values detected
Cyberattack Simulation	Trigger Replay Attack/ Data Spoofing via simulator.	System should detect malicious packets and raise alerts.	Attack detected
Mitigation Control	Apply "Block Device" or "Quarantine Device" option in dashboard.	Device communication should be blocked/quarantined instantly.	Action performed successfully
Web Dashboard Visualization	Open Streamlit dashboard to check real-time visualization.	Live charts and logs should be updated with sensor values and alerts.	Displayed properly
Alert System	Send unauthorized access or anomaly trigger.	Alerts should notify admin via dashboard, Telegram, or email.	Alert generated
Response Time	Measure time from anomaly injection to alert.	Alert should trigger in ≤ 2 seconds.	Triggered in 2 seconds
Alert Logging	Check logs after multiple alerts are triggered.	Alerts should be stored with type, timestamp, and status.	Logs maintained
System Resilience	Run the system under continuous monitoring with attacks enabled.	System should maintain stable operation with minimum downtime.	System remained stable

Table 7.3 shows testing outlines functional and security validations for the proposed IoT healthcare system, covering data acquisition, anomaly detection, cyberattack response, and dashboard performance. Results confirm accurate sensing, timely alerts, and robust system stability under real-time operational conditions.

CHAPTER 8

RESULT AND DISCUSSION

1. Snapshots

1.1. Hardware Prototype Implementation

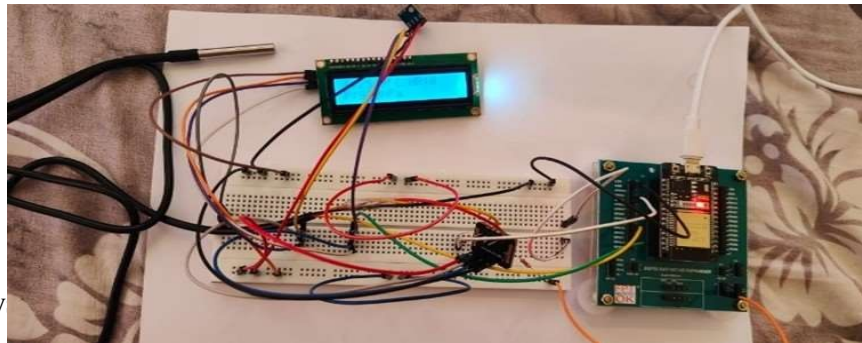


Figure 8.1 Sy... D Sensors As

seen in figure 8.1, The prototype setup uses an ESP32 microcontroller on a breadboard to manage medical IoT sensors: MAX30102 (heart rate, SpO2, DS18B20 (body temperature), and BMP180 (ambient temperature/pressure). It displays vital signs locally on an LCD and transmits the captured data to an external dashboard.

1.2. Real-Time Sensor Data Collection

BMP180 Temp: 27.30 C, Pressure: 909.82 hPa	DS18B20 Temp: 23.81 C	HeartRate IR: 12640	BPM: 0
BMP180 Temp: 27.30 C, Pressure: 909.81 hPa	DS18B20 Temp: 29.87 C	HeartRate IR: 12579	BPM: 0
BMP180 Temp: 27.30 C, Pressure: 909.81 hPa	DS18B20 Temp: 29.44 C	HeartRate IR: 12550	BPM: 0
BMP180 Temp: 27.30 C, Pressure: 909.78 hPa	DS18B20 Temp: 29.62 C	HeartRate IR: 12554	BPM: 0
BMP180 Temp: 27.30 C, Pressure: 909.84 hPa	DS18B20 Temp: 29.75 C	HeartRate IR: 12571	BPM: 0
BMP180 Temp: 27.30 C, Pressure: 909.79 hPa	DS18B20 Temp: 29.56 C	HeartRate IR: 12500	BPM: 0
BMP180 Temp: 27.30 C, Pressure: 909.76 hPa	DS18B20 Temp: 29.81 C	HeartRate IR: 13475	BPM: 0
BMP180 Temp: 27.30 C, Pressure: 909.77 hPa	DS18B20 Temp: 29.69 C	HeartRate IR: 16397	BPM: 0
BMP180 Temp: 27.30 C, Pressure: 909.77 hPa	DS18B20 Temp: 29.69 C	HeartRate IR: 58476	BPM: 0
BMP180 Temp: 27.30 C, Pressure: 909.80 hPa	DS18B20 Temp: 29.56 C	HeartRate IR: 109763	BPM: 0
BMP180 Temp: 27.40 C, Pressure: 909.81 hPa	DS18B20 Temp: 29.37 C	HeartRate IR: 107756	BPM: 0
BMP180 Temp: 28.60 C, Pressure: 909.77 hPa	DS18B20 Temp: 29.69 C	HeartRate IR: 108731	BPM: 0
BMP180 Temp: 29.90 C, Pressure: 909.87 hPa	DS18B20 Temp: 29.19 C	HeartRate IR: 109086	BPM: 0
BMP180 Temp: 30.80 C, Pressure: 909.84 hPa	DS18B20 Temp: 29.50 C	HeartRate IR: 109201	BPM: 0
BMP180 Temp: 31.30 C, Pressure: 909.84 hPa	DS18B20 Temp: 29.19 C	HeartRate IR: 108964	BPM: 0
BMP180 Temp: 31.40 C, Pressure: 909.82 hPa	DS18B20 Temp: 29.00 C	HeartRate IR: 4089	BPM: 0
BMP180 Temp: 31.00 C, Pressure: 909.83 hPa	DS18B20 Temp: 29.00 C	HeartRate IR: 4298	BPM: 0
BMP180 Temp: 30.80 C, Pressure: 909.80 hPa	DS18B20 Temp: 29.12 C	HeartRate IR: 4464	BPM: 0
BMP180 Temp: 30.60 C, Pressure: 909.82 hPa	DS18B20 Temp: 29.50 C	HeartRate IR: 100287	BPM: 0
BMP180 Temp: 30.50 C, Pressure: 909.84 hPa	DS18B20 Temp: 29.87 C	HeartRate IR: 93788	BPM: 0
BMP180 Temp: 31.30 C, Pressure: 909.84 hPa	DS18B20 Temp: 29.87 C	HeartRate IR: 93226	BPM: 0
BMP180 Temp: 31.70 C, Pressure: 909.87 hPa	DS18B20 Temp: 30.12 C	HeartRate IR: 107612	BPM: 0
BMP180 Temp: 32.00 C, Pressure: 909.82 hPa	DS18B20 Temp: 30.56 C	HeartRate IR: 108266	BPM: 0
BMP180 Temp: 32.10 C, Pressure: 909.85 hPa	DS18B20 Temp: 30.56 C	HeartRate IR: 3274	BPM: 0
BMP180 Temp: 31.70 C, Pressure: 909.83 hPa	DS18B20 Temp: 30.62 C	HeartRate IR: 3323	BPM: 0
BMP180 Temp: 31.40 C, Pressure: 909.81 hPa	DS18B20 Temp: 30.56 C	HeartRate IR: 3559	BPM: 0

Figure 8.2 Live Results and Readings

In figure 8.2, The ESP32 gathers and cleans vital sign data (heart rate, SpO2, etc.) from sensors, securing it with a timestamp, ID, and nonce. This structured data is then securely sent to Firebase using TLS and HMAC. Doctors view the real-time vitals on a Streamlit dashboard with low latency.

1.3. Anomaly Detection



Figure 8.3 AI Anomaly Detection Graph

In figure 8.3, The Isolation Forest algorithm detects unusual patterns in real-time sensor data (heart rate, SpO₂, etc.) by learning from past normal and attack data. It assigns an unusualness score to incoming data; sudden, significant deviations are flagged as anomalies. This helps differentiate normal physiological changes from cyberattacks. The model is constantly updated to ensure early and accurate detection, strengthening the system's resilience.

Recent Device Log

	device_id	heart_rate	temperature	blood_pressure	battery	timestamp	attack	attack_type	ml_flag	alerts
19	HRM001	226	38.200000	195	98	2025-11-03 21:48:10	☒	Data Spoof	☒	Anomalous Heart Rate, High Frequency Anomaly, ⚠️ ML Model Detected Anomaly, Known Attack Simulated, Device Operating Outside Permitted Hours
20	HRM001	39	38.500000	71	55	2025-11-03 21:48:11	☒	Data Spoof	☒	Anomalous Heart Rate ⚠️ ML Model Detected Anomaly, Known Attack Simulated, Device Operating Outside Permitted Hours
21	HRM001	208	35.500000	186	60	2025-11-03 21:48:12	☒	Data Spoof	☒	Anomalous Heart Rate ⚠️ ML Model Detected Anomaly, Known Attack Simulated, Device Operating Outside Permitted Hours
22	HRM001	199	39.200000	87	61	2025-11-03 21:48:14	☒	Data Spoof	☒	Anomalous Heart Rate ⚠️ ML Model Detected Anomaly, Known Attack Simulated, Device Operating Outside Permitted Hours
23	HRM001	36	34.100000	199	91	2025-11-03 21:48:15	☒	Data Spoof	☒	Anomalous Heart Rate ⚠️ ML Model Detected Anomaly, Known Attack Simulated, Device Operating Outside Permitted Hours

⚠️ Anomalous Heart Rate, ⚠️ ML Model Detected Anomaly, Known Attack Simulated, Device Operating Outside Permitted Hours

💡 Suggestion: Consider Blocking or Quarantining this device.

🔍 Check 'Run Simulation' in the sidebar to begin.

Figure 8.4 Event Logs

In figure 8.4 show the Patient Logs are also displayed in the Dashboard so that quick response can be seen.



Figure 8.5 Final Dashboard Display

In figure 8.5, The Streamlit dashboard acts as an interface for a Firebase cloud database, providing real-time visualizations of patient vitals (heart rate, SpO2 temperature, pressure) via interactive charts. It also displays cybersecurity analytics like heatmaps for attack effects and anomaly counts. The dashboard includes an event log and mitigation controls (blocking/isolating devices), allowing healthcare professionals to make immediate assessments of both medical conditions and security risks.

2. Discussion

This work establishes a foundation for secure smart healthcare by linking medical monitoring with cybersecurity. The system utilizes small sensors and an ESP32 board to collect real-time bodily parameters (heart rate, temperature, SpO2) while simultaneously identifying web threats. It provides a real-time view of both patient health status and data link security. Smart checks identify abnormal data or fake logs, enhancing system safety and reliability.

By using a live-logged cloud with safety protocols, the system ensures immutability—no one can alter past records—which significantly enhances trust. AI monitoring enables the detection of both health and hacking threats. The intuitive dashboard and graphs offer clear views of system trends and alerts.

Ultimately, this integrated setup functions robustly within real-world environments, protecting health data from both technical failures and cyber risks. Experiments confirm that this solution, featuring real-time monitoring and secure ledger rules, provides a comprehensive and scalable model for smart healthcare, ready for future expansion.

CHAPTER 9

CONCLUSION, APPLICATIONS AND FUTURE WORK

1. Conclusion

This project successfully designed and tested a complete cybersecurity risk assessment framework for MIIOT networks, integrating physical hardware, a secure cloud, and a real-time security dashboard.

The core system uses an ESP32 device and sensors (MAX30102, DS18B20, BMP180) to gather patient vitals, securing the transmission to Google Firebase Cloud with TLS encryption and HMAC. The primary innovation is the dual-layer detection engine on the Streamlit dashboard, which combines rule-based detection for known threats with the Isolation Forest ML algorithm for novel anomalies.

Rigorous testing using a Dynamic Risk Assessment (DRA) framework confirmed the system's resilience against simulated cyberattacks. The dashboard successfully identified and logged all threats in real-time and provided mitigation controls (blocking/quarantining). This lightweight, portable, and secure framework bridges patient monitoring and active cybersecurity defense, offering a viable solution for modern healthcare.

2. Application

The integrated and cost-effective nature of this system makes it suitable for a wide variety of healthcare applications:

- i. **Remote Patient Monitoring:** The system is ideal for home use, allowing healthcare providers to monitor patients' vital signs in real-time from a distance, enabling faster diagnosis and intervention.
- ii. **Hospital Ward Monitoring:** The portable and lightweight design allows for easy deployment in hospital wards as a supplementary monitoring system, especially for non-critical patients, helping to offload hospital resources.
- iii. **Telemedicine Services:** The framework serves as a reliable data source for telemedicine consultations, providing doctors with a complete and secure view of a patient's health status during a remote visit.
- iv. **Cybersecurity Testbed:** The entire project, including the attack simulation scripts and the DRA framework, serves as a valuable academic and research testbed for evaluating new MIIOT security protocols, detection algorithms, or defense-in-depth strategies.

3. Future Scope of The Project

Future development should focus on enhancing resilience and compliance through several key areas:

- i. **On-Device AI (TinyML):** Deploying a lightweight anomaly detection model directly onto the ESP32 to enable "offline" detection and local patient alerts even when Wi-Fi is unavailable.
- ii. **Hybrid Connectivity:** Integrating a 4G/LTE or LoRaWAN module as a fallback to ensure continuous data transmission if the primary Wi-Fi connection fails.
- iii. **Adaptive Machine Learning:** Implementing an automated pipeline to periodically retrain the Isolation Forest model on new, validated data from Firebase, allowing the system to adapt to changes in patient health or novel cyberattacks.
- iv. **Blockchain for Data Integrity:** Logging alert hashes and critical events to a lightweight blockchain to create a fully immutable, tamper-proof audit trail for forensic analysis and regulatory compliance.
- v. **Formal Compliance and Hardening:** Formally testing the system against healthcare-specific regulatory standards, such as HIPAA and NISTIR 8228, for clinical certification.

CO-PO MAPPING

i. PO/PSO Covered for Major Project with Justification

CO1	Apply engineering knowledge & research methods to solve complex problems.
CO2	Design & develop feasible, innovative engineering solutions.
CO3	Plan, execute & manage a project within defined timelines.
CO4	Communicate technical findings effectively.
CO5	Work effectively individually & in a multidisciplinary team.
CO6	Exhibit awareness of ethical, societal & environmental implications.

ii. CO-PO Mapping

COs	PO 1	PO 2	PO 3	PO 4	PO 5	PO 6	PO 7	PO 8	PO 9	PO10	PO11	PS01	PS02
CO 1	3	3		3	3								2
CO 2	3	3	3		3						2	2	2
CO 3			3		3				3	2	3	3	2
CO 4								2		3			2
CO 5					2			2	3	2	2	2	2
CO 6				2		3	3	3					2
Avg	2.0	2.0	3.0	2.5	2.75	3.0	3.0	2.25	3.0	2.33	2.33	2.33	2.0

iii. CO-PO/PSO Justification Table

CO	PO/PSO	Justification
CO 1	PO1, PO2, PO4, PO5, PS02	Uses engineering knowledge, analysis, research, and AI tools for problem solving.
CO 2	PO1, PO2, PO3, PO5, PS01, PS02	Designs AI-based solutions using engineering design, tools, and software/AI skills.
CO 3	PO3, PO5, PO9, PO10, PO11, PO12, PS01, PS02	Involves project execution, teamwork, communication tools, and management skills.
CO 4	PO8, PO10, PO12	Communicates findings ethically and clearly through reports and presentations.

CO 5	PO5, PO8, PO9, PO10, PO11, PO12, PSO1, PSO2	Demonstrate teamwork, ethics, communication, and tool usage across project tasks.
CO 6	PO4, PO6, PO7, PO8, PO12, PSO2	Considers ethical, societal, environmental, and research-based aspects in solutions.

Sustainable Development Goals

1. R.M. Czekster, T.Webber, L.B. Furstenau, and C.Marcon, "Dynamic risk assessment approach for analysing cyber security events in medical IoT-networks," *Internet of Things*, vol. 29, p. 101437, 2025.
[Online]. Available: <https://doi.org/10.1016/j.iot.2024.101437>
2. A. Si-Ahmed, M.A.Al-Garadi, and N.Boustia, "Survey of Machine Learning Based Intrusion Detection Methods for Internet of Medical Things," *arXiv preprint*, 2023.
[Online]. Available: <https://doi.org/10.48550/arXiv.2306.03729>
3. T. S. AlSalem, M. A Almaiah, and A.Lutfi, "Cybersecurity Risk Analysis in the IoT: A Systematic Review," *Electronics*, vol. 12, no. 18, p. 3958, 2023.
[Online]. Available: <https://doi.org/10.3390/electronics12183958>
4. Z. ElSayed, A. Abdelgawad, and N. Elsayed, "CryptoDNA: A Machine Learning Paradigm for DDoS Detection in Healthcare IoT, Inspired by Cryptojacking Prevention Models," *arXiv preprint*, 2025.
[Online]. Available: <https://doi.org/10.48550/arXiv.2402.06340>
5. J. Hemmes, S. Fulton, J. Dressler, and S. Kirkman, "Trust and Risk Assessment in IoT Networks," *Computer Networks and Communications*, vol. 1, no. 1, pp. 181–199, 2023.
[Online]. Available: <https://doi.org/10.48550/arXiv.2303.14153>
6. A. Alshahrani, M. Alharthi, and A. Alghamdi, "Towards Design and Development of Security Assessment Framework for Internet of Medical Things," *Applied Sciences*, vol. 12, no. 16, p. 8148, 2022.
[Online]. Available: <https://doi.org/10.3390/app12168148>
7. O. Cheikhrouhou, K. Mershad, F. Jamil, R. Mahmud, A. Koubaa, and S. R. Moosavi, "A Lightweight Blockchain and Fog-Enabled Secure Remote Patient Monitoring System," *Internet of Things*, vol. 22, p. 100691, 2023.
[Online]. Available: <https://doi.org/10.1016/j.iot.2023.100691>
8. D. Klonoff, "Patient Involvement in Cybersecurity for Medical Devices," *Journal of Diabetes Science and Technology*, vol. 9, no. 6, pp. 1350–1355, 2015.
[Online]. Available: <https://doi.org/10.1177/1932296815610559>

9. T. P. Dover, “Evaluating Medical IoT (MIoT) Device Security using NISTIR-8228 Expectations,” *arXiv preprint*, 2021.
[Online]. Available: <https://doi.org/10.48550/arXiv.2107.11099>
10. J. R. C. Nurse, S. Creese, and D. De Roure, “Security Risk Assessment in Internet of Things Systems,” in *Living in the Internet of Things: Cybersecurity of the IoT – 2018*, London, U.K., 2018, pp. 1–6.
[Online]. Available: <https://doi.org/10.1049/cp.2018.0043>

ANNEXURE A

Paper Publication Certificate



Protatva-2025 Project Exhibition Certificate





ANNEXURE B

KSCST Declaration

DECLARATION
(From Project Students)
(To scan this page and enclose in the project proposal)

We, the project team hereby declare that the details enclosed in the project proposal (Title of the Project: **CYBERSECURITY RISK ASSESSMENT IN MEDICAL IOT(MIOT) NETWORKS**, Branch: **Computer Science and Engineering**, College: **AMC Engineering College**) are true and correct to the best of our knowledge and belief. We undertake to inform the Karnataka State Council for Science and Technology (KSCST) of any changes to the project title or team members' names immediately through our project guide or the SPP Coordinator of our institution.

Additionally, we declare that the project work is original and not a result of copying or purchasing. We are committed to completing the project independently, with support from our faculty and project guide, while utilizing the facilities provided by the college. We will not engage in plagiarism and pledge to be sincere and dedicated in executing and completing the project as proposed.

We understand that if any of the above information is found to be false, untrue, or misleading, we may be held liable. We authorize the sharing of the project information contained in this proposal with KSCST, Bengaluru.

We acknowledge that the project team must exhibit and demonstrate the project, participate in the mid-term evaluation of sanctioned projects, and engage with experts. Additionally, we must exhibit the project at the Annual State-Level Poster Presentation and Exhibition, if selected. Should our team fail to participate in the mid-term evaluation or the Annual Exhibition (if selected), we understand that the supported project funds will be returned to KSCST.

We also enclose the endorsement form for KSCST, Bengaluru.

Name of the students with USN No. 1. Vaishnavi (1AM22CS216) 2. Shiva Charan Reddy Kota (1AM22CS184) 3. Shubham Jaiswal (1AM22CS191) 4. Umme Barira (1AM22CS212) <i>M. Deshpande</i> 11/12/21 Dr. Meena Deshpande (Name & Signature of SPP Coordinator with Seal) Name: Dr. Meena Deshpande Email id: meena.deshpande@amceducation.in Contact No.: 9945060056	Signature with date <i>[Signature]</i> Shivacharan <i>[Signature]</i> [Signature] Name and signature of the Project Guide with Seal: Name: Mrs. Snigdha Kesh Email id: snigdha.kesh@amceducation.in Contact No.: 9916954133
---	---

KSCST: Student Project Programme: 49th series: 2025-2026

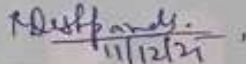
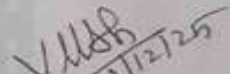
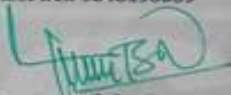
KSCST Endorsement






ENDORSEMENT
(From College, endorsement to be taken in the Institution / Department Letter head)
(To scan this page and enclose in the project proposal)

This is to certify that 1) Ms. Vaishnavi (1AM22CS216), 2) Mr. Shiva Charan Reddy Kota (1AM22CS184), 3) Mr. Shubham Jaiswal (1AM22CS191), 4) Ms. Umme Barira (1AM22CS212), are bonafide student(s) of Department of Computer Science and Engineering, in the degree program of our institution. If the project proposal submitted by these students under the 49th series of Student Project Programme is selected by KSCST, we will provide the requisite laboratory / Computer / infrastructure support in our college / Institution. Further we also take necessary steps to see that the project team will exhibit / demonstrate their project in the mid-term evaluation of project and in the Annual State-Level Poster Presentation and Exhibition (if selected). If the student team fails to send the completed project report or fails to attend the evaluation in mid-term evaluation of sanctioned projects or fails to attend the Annual State-Level Poster Presentation and Exhibition (if selected), the supported project amount will be returned to KSCST.

 (Name & Signature of Project Guide with Seal)	 (Name & Signature of SPP Coordinator with Seal)	 (Signature of HOD with Seal)
Name: Mrs. Snigdha Kesh Email id: snigdha.kesh@amceducation.in Contact No.: 9916954133	Name: Dr. Meena Deshpande Email id: meena.deshpande@amceducation.in Contact No.: 9943060056	Name: Dr. V. Marceswari Email id: marresh.prasanna@amceducation.in Contact No.: 9845198389
		 (Signature of the Principal with Seal) Name: Dr. Yuvaraju B N Email id: principal@amceducation.in Contact No.: 9483101919 PRINCIPAL AMC ENGINEERING COLLEGE BENGALURU - 560 083

KSCST: Student Project Programme: 49th series: 2025-2026
 18th K.M. Kalkere, Bannerghatta Road, Bangalore - 560083. AMC Engineering College, Admissions: 9902044113, Office: 9902044114, Telefax: 080-27828636. Email: principal@amcec.edu.in | Website: www.amcgroup.edu.in